



UNIVERSITÀ
DEGLI STUDI
DI TRIESTE



APPLIED DATA SCIENCE &
ARTIFICIAL INTELLIGENCE



Application of Large Language Models

Addressing Real-World Challenges

Giovanni Pinna

3 March 2026

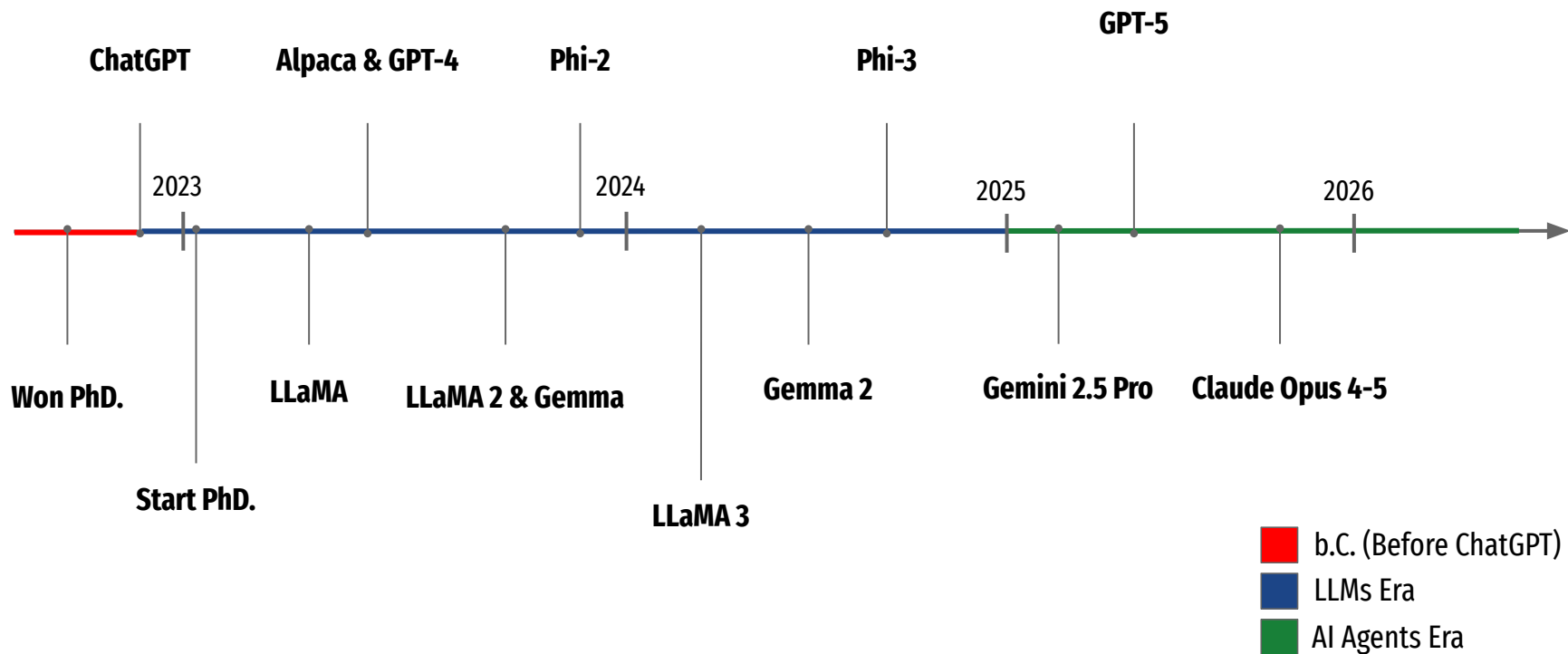
Introduction

The advent of Large Language Models (LLMs) represents a fundamental turning point in artificial intelligence and computer science.

In less than a decade, **LLMs have become sophisticated systems capable of performing tasks that, until recently, were considered the exclusive domain of human cognition.** Systems such as GPT-4 [7], Claude [32], and LLaMA [33] demonstrate unprecedented capabilities in text generation, translation, code generation, and reasoning.

This technology can democratize access to information and domains that were previously reserved only for specialized experts [13] (e.g., law, medicine, software engineering...).

Timeline



Problem

This transformation goes far beyond incremental progress.

It forces researchers to **rethink fundamental assumptions about artificial intelligence, evaluation methodologies, and the societal implications** of these systems, as well as the recognition of current limitations and associated risks [14].

LLMs, despite their impressive capabilities, remain imperfect.

Problem

1. **Fluency $\neq$ Correctness:** LLMs produce extremely fluent text that masks subtle errors. (E.g., in the legal domain, an apparently correct summary may contain inaccuracies; in code, syntactically valid functions may fail on edge cases.)
2. **Inadequate Metrics:** binary and/or superficial metrics fail to capture the nuance and complexity of the outputs.
3. **Unsustainable Costs:** the computational cost of LLM inference, especially in agentic workflows where a single task may require hundreds of calls, poses significant challenges to environmental sustainability.

Research questions

The main objective is to develop a comprehensive understanding of how to evaluate, improve, and sustainably deploy LLM-based systems across different application domains.

RQ1: *How can we develop more accurate and meaningful evaluation methodologies for LLM outputs that capture semantic correctness and practical utility beyond superficial metrics?*

RQ2: *To what extent can LLMs democratize access to specialized knowledge in the legal and software engineering domains, and what are the limitations and risks of current systems in these contexts?*

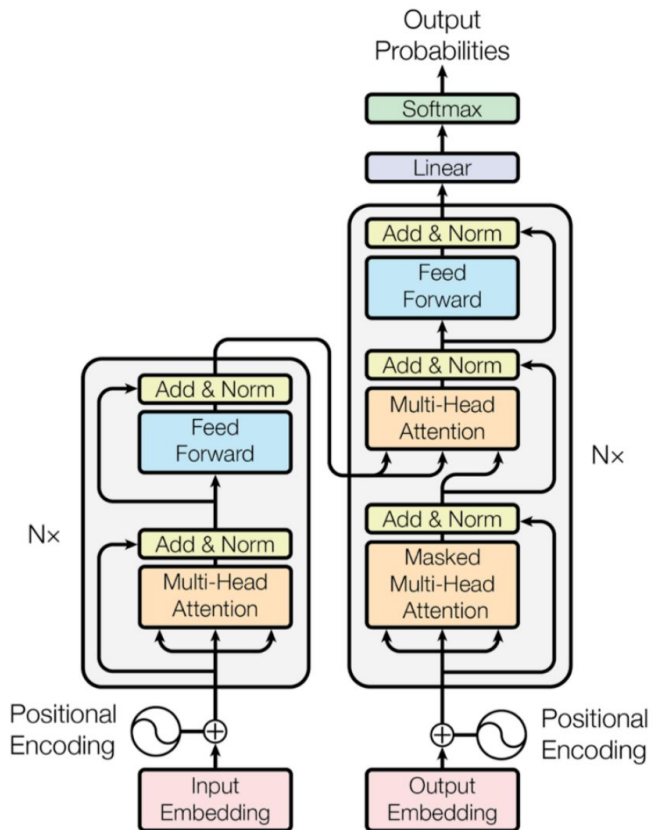
Research questions

RQ3: *What post-processing and hybrid approaches can improve the reliability of LLM outputs in high-stakes applications?*

RQ4: *How can we develop and deploy LLM-based systems while minimizing their environmental footprint and ensuring sustainable resource usage?*

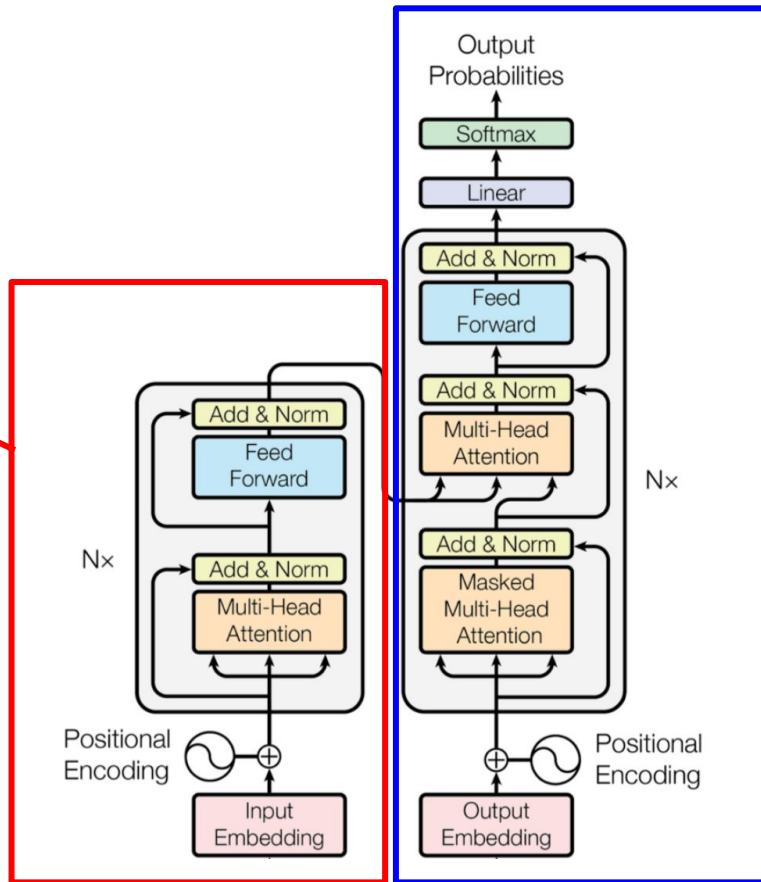
Background & Terminology

Transformer



Transformer

Encoder
(BERT)



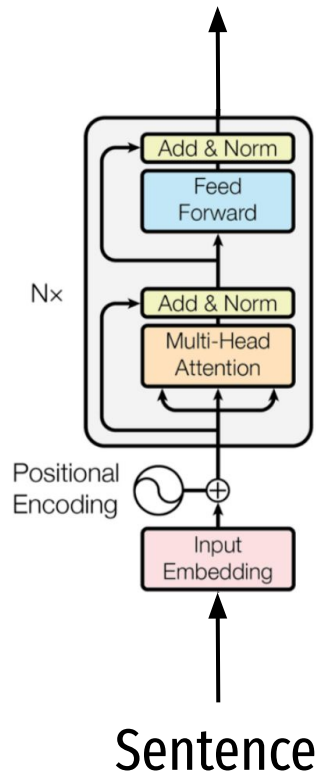
Decoder
(GPT)

Encoder

The encoder generates a vector in a dimensional space (typically 384 or 768 dimensions).

The vector is created to represent the semantics of the sentence in n-dimensional space.

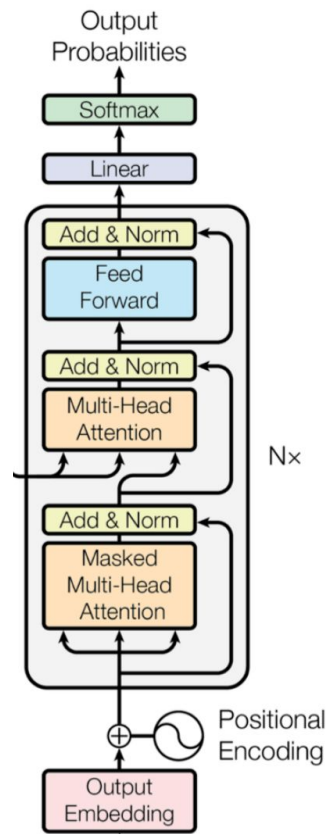
$[0.1, 0.02, 0.04, 0.7, \dots]$



Decoder

At each step, it generates a token, adds it to the output sequence, and uses it as input for the next step. Generation continues until an end-of-sequence token (EOS) is produced.

LLMs are decoder-only architectures.



LLM

LLM: a Transformer-based model that **generates text by predicting the next token**, trained on billions of data points.

Pre-training: the model is trained to predict the next token on massive amounts of text (books, web, code), learning grammar, knowledge, and reasoning abilities in an unsupervised way.

Fine-tuning: the raw model is refined with high-quality conversation examples (SFT) and human feedback, making it useful, safe, and able to follow instructions.

Token: the **smallest unit of text processed by the model** it can be a word, part of a word, or a single character, depending on the tokenizer.

LLM

Context Window: the maximum context size the model can "keep in mind" in a single conversation or request. If you exceed the limit, the model no longer sees the oldest parts and is no longer reliable.

The LLM has also other parameters such as:

- temperature
- top_p
- max_token
- ...

Genetic Algorithm (GA)

Genetic Algorithm (GA) is an optimization and search method inspired by natural selection mechanisms, operating on a population of candidate solutions encoded as individuals by applying selection, crossover, and mutation operators to iteratively evolve increasingly fit solutions to maximize (or minimize) an objective function.

GP, GI, and GE

Genetic Programming (GP) is an optimization technique inspired by natural evolution, applied to the evolution of computer programs. The quality of each program is measured by a fitness function — for example, the number of test cases passed.

Genetic Improvement (GI): is a specialized application of GP: instead of evolving programs from scratch, it improves existing code.

Grammatical Evolution (GE): a formal grammar (e.g., a BNF grammar) constrains the search space, ensuring that all generated programs are syntactically valid.

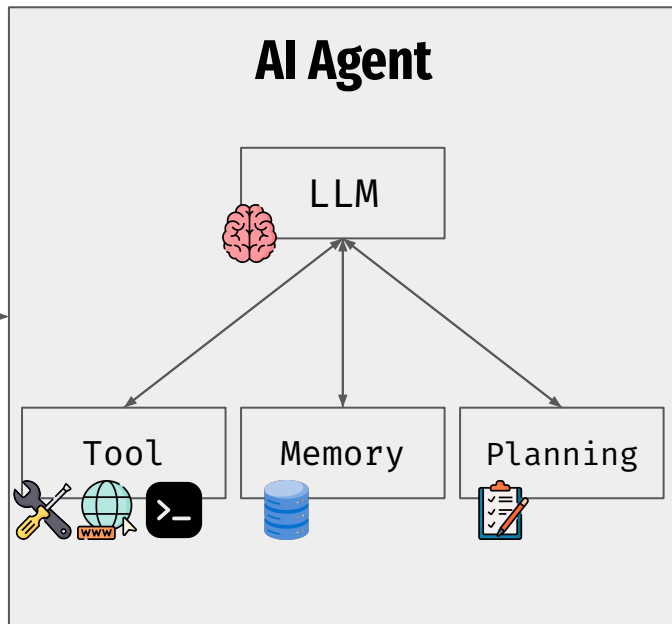
AI Agent

AI Agent is an autonomous computational entity that perceives the state of an environment through inputs (observations), selects actions based on a decision function (policy or internal model), and executes those actions on the environment with the goal of maximizing a utility function or satisfying defined performance criteria.

Agent step: a single unit of "*action + decision*" that the agent performs while interacting with an environment.

AI Agent

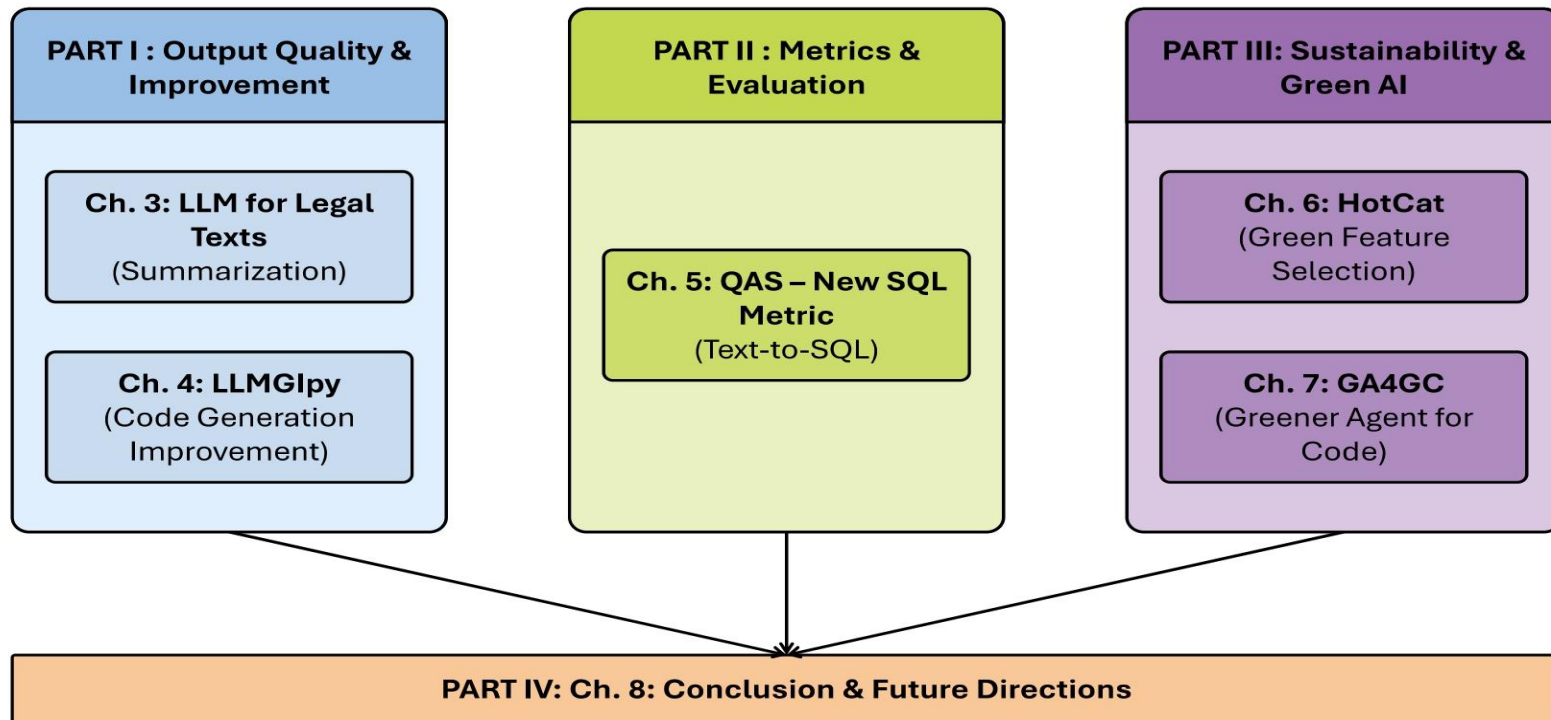
Request



“Solution”

Environment

Presentation Structure



I

Output Quality and Improvement

LLM for legal texts

G. Pinna, D. Tugnoli, M. Bartole, L. Manzoni, A. De Lorenzo, "From Courts to Comprehension: Can LLMs Make Judgments More Accessible?," *IEEE/WIC International Conference on Web Intelligence and Intelligent Agent Technology (WI-IAT)*, pp. 297–304, IEEE, 2024.

Law comprehension (The problem)

In a rule-of-law society, **legal regulations profoundly impact citizens' daily lives.** However, **understanding these laws is difficult for those without legal knowledge** [36].

To address this problem, many researchers have used NLP [37, 38, 39], but the **evaluation of the quality of generated text on people without legal knowledge has not yet been explored** [40].

- **Judgment:** act used to designate the measure that resolves a dispute.
- **Massima:** is an extrapolation of some concepts from the judgments (**needed strong legal skills, extremely time-consuming and costly to produce**).

Law comprehension (Pipeline)

Our proposal:

- **Indirect assessment of the quality of the summaries based on how comprehensible they were to the target audience.**
 - Survey: legal expert, created comprehension questions about judgments.
- Generating judgment summaries with LLMs and LLMs + fine-tuning.
- Put questionnaire on web app to involve as many people as possible (75 participants).

Law comprehension (Data & LLMs & Survey)

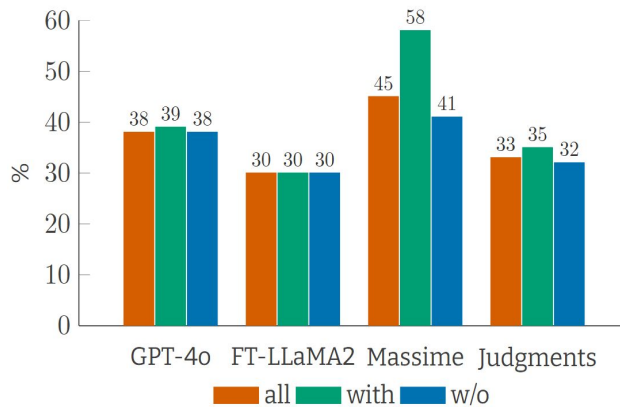
Data: Open data provided by the constitutional court. Judgments and related *massime* from 1956 to the present. (about 18k texts)[41].

LLMs: LLaMA 2 7B fine-tuned & GPT-4o [7] (through API).

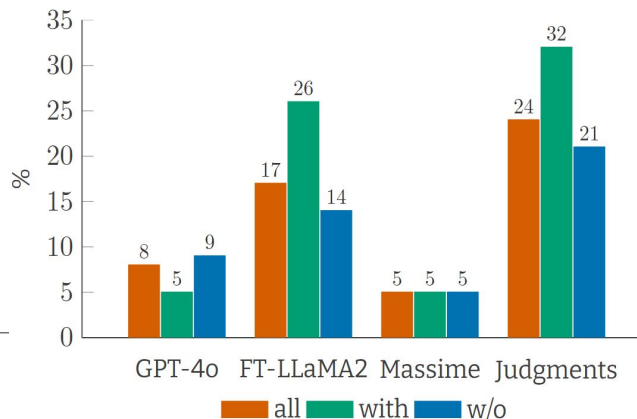
Survey: Select 25 judgments from 2000 to 2023. For each judgment 3 questions with 4 possible answers + “I don't know” or skip text.

Texts presented to each user: 1 *massima*, 1 judgment, 2 summaries generated.

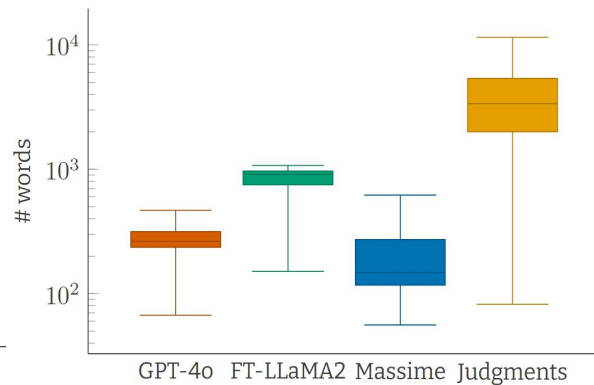
Law comprehension (Results)



Percentage of correct answers for each type of text for all participants (**all**), for participants with legal knowledge (**with**), and for people without legal knowledge (**w/o**).



Percentage of skipped answers for each type of text for all participants (**all**), for participants with legal knowledge (**with**), and for people without legal knowledge (**w/o**).



Length (in number of words) **of the texts** for each category. The boxes show the 25-th and 75-th percentiles, while the whiskers denote the minimum and maximum. Vertical axis is in *logarithmic* scale.

Law comprehension (Conclusion)

1. Human experts excel at balancing precision and accessibility;
2. GPT-4o offers a scalable and cheaper but risky alternative, prone to generating plausible but inaccurate content that represents a significant danger in legal contexts;
3. Open-source models like fine-tuned LLaMA remain inadequate due to architectural limitations (e.g., context window), providing no practical utility.

Human-centered evaluation is essential in high-stakes domains: automatic metrics do not detect subtle hallucinations.

LLMGlpy

G. Pinna, D. Ravalico, L. Rovito, L. Manzoni, A. De Lorenzo, "Enhancing Large Language Models-Based Code Generation by Leveraging Genetic Improvement," *European Conference on Genetic Programming (EuroGP)*, LNCS 14631, pp. 108–124, Springer, 2024.

G. Pinna, D. Ravalico, L. Rovito, L. Manzoni, A. De Lorenzo, "Improving LLM-Generated Code via Genetic Improvement: A Summary of Recent Advances," *5th National Conference on Artificial Intelligence*, CINI, 2025.

G. Pinna, D. Ravalico, L. Rovito, L. Manzoni, A. De Lorenzo, "Exploring the Effect of Genetic Improvement for Large Language Models-Generated Code," *SN Computer Science*, vol. 6, no. 7, p. 760, Springer, 2025.

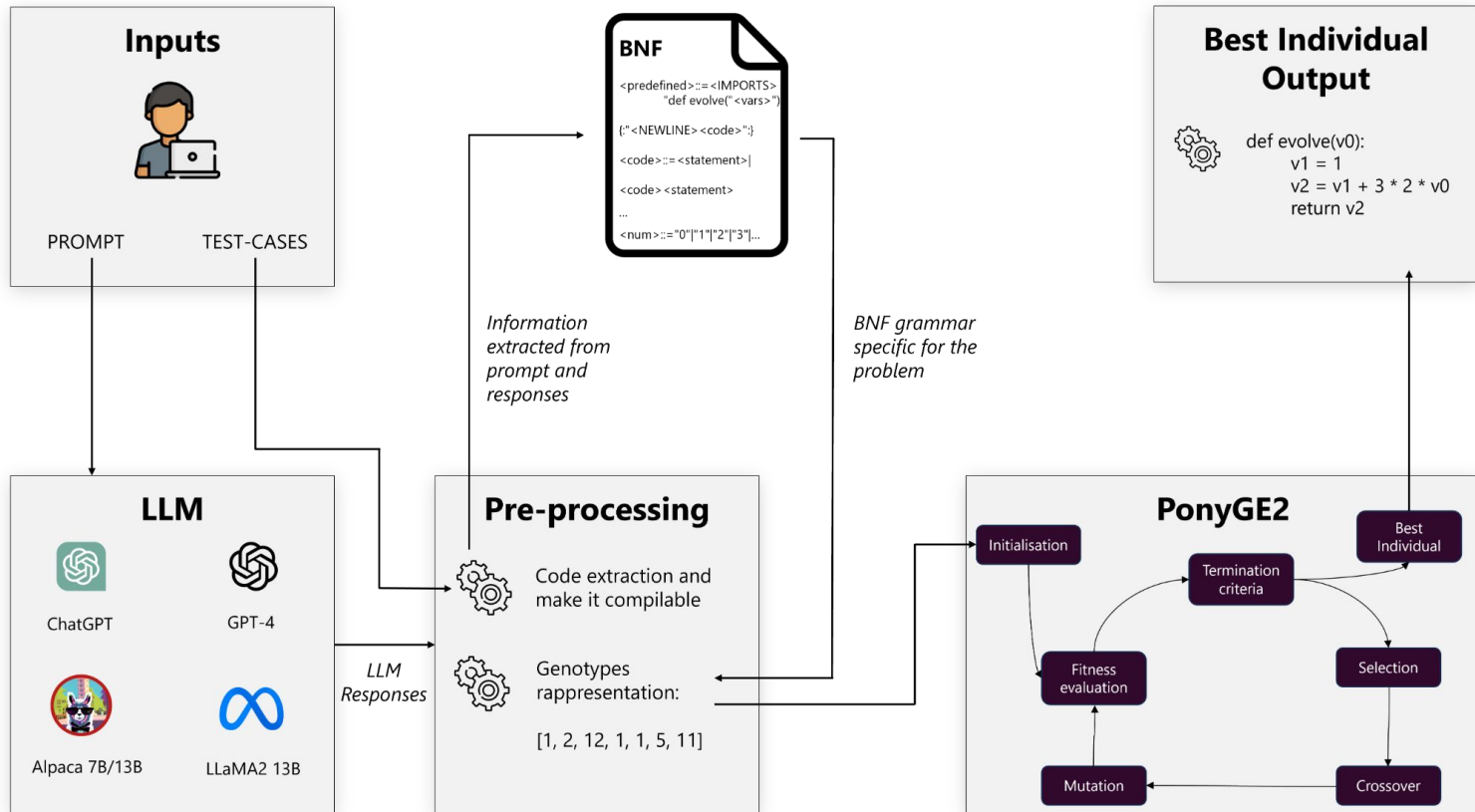
LLMGlpy (The problem)

In recent years, large language models (LLMs) have improved the state of the art in many natural language processing (NLP) tasks.

It has been shown that even the most effective **LLMs often fail to produce correct code** [43, 44, 45].

To address this problem, **we propose an evolutionary approach based on genetic improvement (GI) to improve the code generated by an LLM** using a set of user-supplied test cases.

LLMGLpy (Pipeline)



LLMGlpy (Inputs & LLMs)

Used the 25 problems of the **PSB2 benchmark** [4, 5]. This benchmark is widely used in genetic programming and genetic improvements.

- Textual description of the problem
- Train/Test-cases (1000)

Selected LLMs:

- Chat-GPT [6]
- GPT-4 [7]
- LLaMA 2 13B [8]
- Alpaca 7B - 13B [9]

LLMGlpy (Self-correction)

Self-correction consists in asking the LLM to correct the code that it generated if it is incorrect (according to our test-cases).

Time out prompt:

```
Your code is too slow. Please, rewrite it and make it correct and more efficient. Remember to insert the necessary modules.
```

Syntax error prompt:

```
Your code contains syntax errors. Please, rewrite it and fix all the syntax errors. Remember to insert the necessary modules.
```

LLMGlpy (Dynamic grammar) 1/3

Request:

Given an integer x, return
"Fizz" if x is divisible by 3,
"Buzz" if x is divisible by 5,
"FizzBuzz" if x is divisible by
3 and 5, and a string version
of x if none of the above
hold.

Response:

Sure! Here it is you programm:

```
def compute(x):  
    if x % 5 and x % 3:  
        return "Fizz"  
    elif x % 5:  
        return "Buzz"  
    else:  
        return "Fizz"
```

This code solve the FizzBuzz Problem.

LLMGlpy (Dynamic grammar) 2/3

Request:

Given an integer `x`, return
"Fizz" if `x` is divisible by 3,
"Buzz" if `x` is divisible by 5,
"FizzBuzz" if `x` is divisible by
3 and 5, and a string version
of `x` if none of the above
hold.

Response:

Sure! Here it is you programm:

```
def compute(x):  
    if x % 5 and x % 3:  
        return "Fizz"  
    elif x % 5:  
        return "Buzz"  
    else:  
        return "Fizz"
```

This code solve the FizzBuzz Problem.

LLMGlpy (Dynamic grammar) 3/3

`<predefined> ::= “def evolve(”<vars>”):{”<NEWLINE><code>”:}”`

...

`<tags> ::= rules`

...

`<FUNC> ::= “”`

`<METHODS> ::= “”`

`<STRINGS> ::= “Fizz” | “Buzz” | “FizzBuzz” | “string version”`

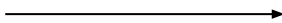
`<var> ::= “v0” | “a0” | “a1” | “a2”`

`<num> ::= “0” | “1” | “2” | “3” | “4” | “5” | “6” | “7” | “8” | “9”`

LLMGlpy (Pre-processing)

Sure! Here it is you programm:

```
def compute(x):  
    if x % 5 and x % 3:  
        return "Fizz"  
    elif x % 5:  
        return "Buzz"  
    else:  
        return "Fizz"
```

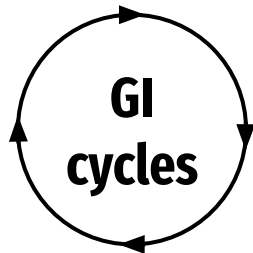


```
def evolve(v0):{:#  
    if v0 % 5 and v0 % 3:{:#{  
        return "Fizz">#:}  
    elif v0 % 5:{:#{  
        return "Buzz">#:}  
    else:{:#{  
        return "Fizz">#:}:}
```

This code solve the FizzBuzz Problem.

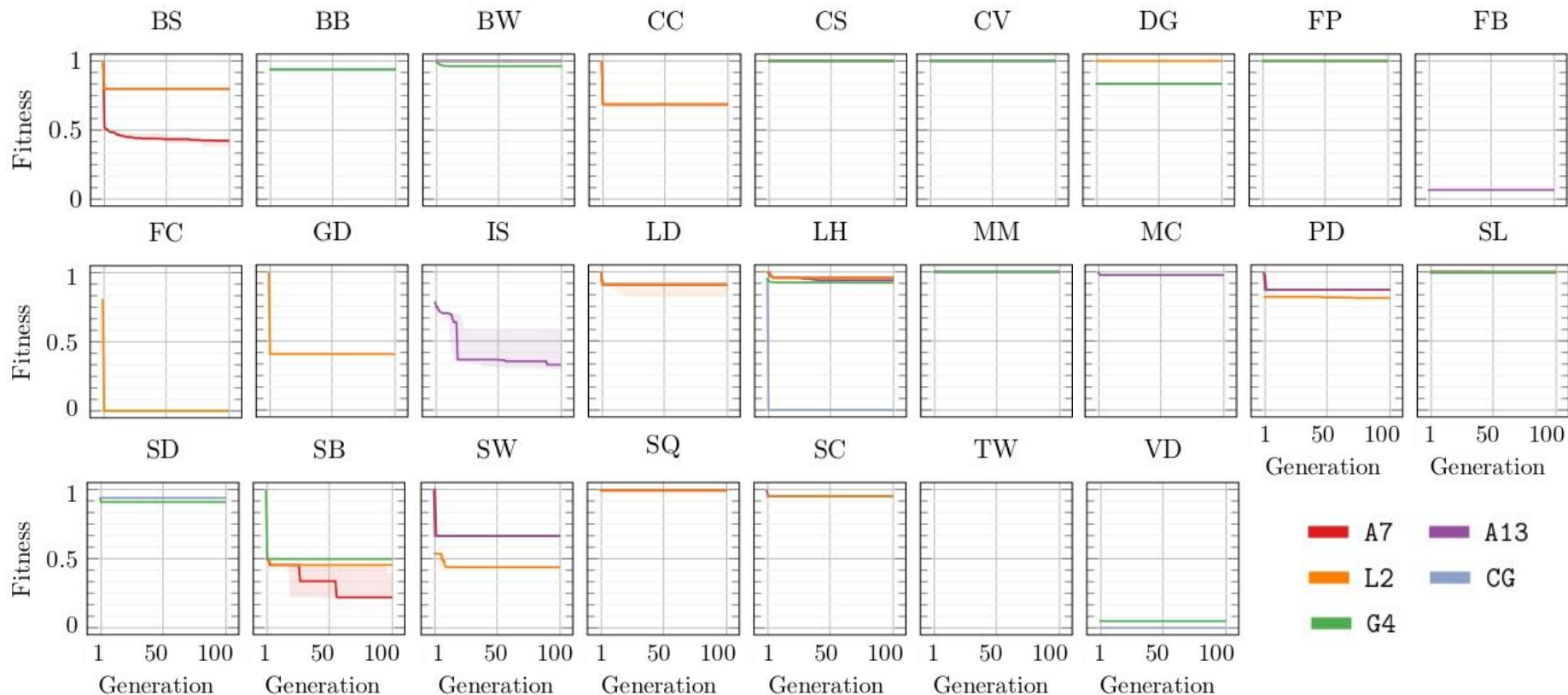
LLMGlpy (Evolution)

```
def evolve(v0):{:#  
    if v0 % 5 and v0 % 3{:{:#  
        return "Fizz"#:}  
    elif v0 % 5{:{:#  
        return "Buzz"#:}  
    else{:{:#  
        return "Fizz"#:}:}
```



```
def evolve(v0):{:#  
    if v0 % 5 and v0 % 3{:{:#  
        return "FizzBuzz"#:}  
    elif v0 % 5{:{:#  
        return "Buzz"#:}  
    elif v0 % 3{:{:#  
        return "Fizz"#:}  
    else{:{:#  
        return str(v0)#:}:}
```

LLMGlpy (Results)



LLMGIpy (Conclusion)

The empirical evaluation on 25 problems and 5 models demonstrates that GI improves code in a statistically significant way, with some cases reaching perfect correctness starting from completely incorrect solutions.

The superiority of GI over self-correction reveals fundamental limitations of current LLMs: they excel at pattern recognition and syntactic generation but lack the systematic reasoning required for debugging and logical refinement.

Dynamic grammar specialization is a key methodological contribution.

II

Metrics and Evaluation

Query Affinity Score (QAS)

QAS (The problem)

There are currently **2 main measures for checking the correctness** of a generated **SQL query against a gold SQL query** (thus correct by definition).

- **Exact match accuracy (EM)**: comparison between two SQL strings [21].
- **Execution accuracy (EX)**: comparison between the two tables obtained from the execution of the two SQL queries [21].

QAS (The problem) 2/7

Exact Match accuracy (EM): comparison between SQL query strings.

```
SELECT T2.MailStreet  
FROM frpm AS T1 INNER JOIN schools AS T2  
ON T1.CDSCode = T2.CDSCode  
ORDER BY T1.`FRPM Count (K-12)` DESC  
LIMIT 1;
```

```
SELECT T2.MailStreet  
FROM frpm AS T1 INNER JOIN schools AS T2  
ON T1.CDSCode = T2.CDSCode  
ORDER BY T1.`FRPM Count (K-12)` DESC  
LIMIT 1;
```

EM = 1



QAS (The problem) 3/7

Exact Match accuracy (EM): comparison between SQL query strings.

```
SELECT T2.MailStreet  
FROM frpm AS T1 INNER JOIN schools AS T2  
ON T1.CDSCode = T2.CDSCode  
ORDER BY T1.`FRPM Count (K-12)` DESC  
LIMIT 1;
```

```
SELECT mailstreet  
FROM schools  
WHERE cdscode IN (  
  SELECT cdscode  
  FROM frpm  
  ORDER BY `FROM Count (K-12)` DESC  
  LIMIT 1);
```

EM = 0



QAS (The problem) 4/7

EXecution accuracy (EX): comparison between tables obtained from executing the SQL queries. Uses *set* equality.

```
SELECT T2.MailStreet
FROM frpm AS T1 INNER JOIN schools AS T2
ON T1.CDSCode = T2.CDSCode
ORDER BY T1.`FRPM Count (K-12)` DESC
LIMIT 1;
```

```
SELECT mailstreet
FROM schools
WHERE cdscode IN (
  SELECT cdscode
  FROM frpm
  ORDER BY `FROM Count (K-12)` DESC
  LIMIT 1);
```

QAS (The problem) 5/7

EXecution accuracy (EX): comparison between tables obtained from executing the SQL queries. Uses *set* equality.

MailStreet
14429 South Downey Avenue

MailStreet
14429 South Downey Avenue

EX = 1



QAS (The problem) 6/7

EXecution accuracy (EX): comparison between tables obtained from executing the SQL queries. Uses *set* equality.

```
SELECT DISTINCT(t1.player_name)
FROM Player AS t1 INNER JOIN
Player_Attributes AS t2 ON
t1.player_api_id = t2.player_api_id
WHERE t2.potential = (
    SELECT MAX(potential)
    FROM Player_Attributes);
```

```
SELECT T1.player_name
FROM player AS T1 INNER JOIN
player_attributes AS T2 ON
T1.player_api_id = T2.player_api_id
WHERE T2.potential = (
    SELECT MAX(potential)
    FROM player_attributes);
```

QAS (The problem) 7/7

EXecution accuracy (EX): comparison between tables obtained from executing the SQL queries. Uses *set* equality.

player_name
Lionel Messi

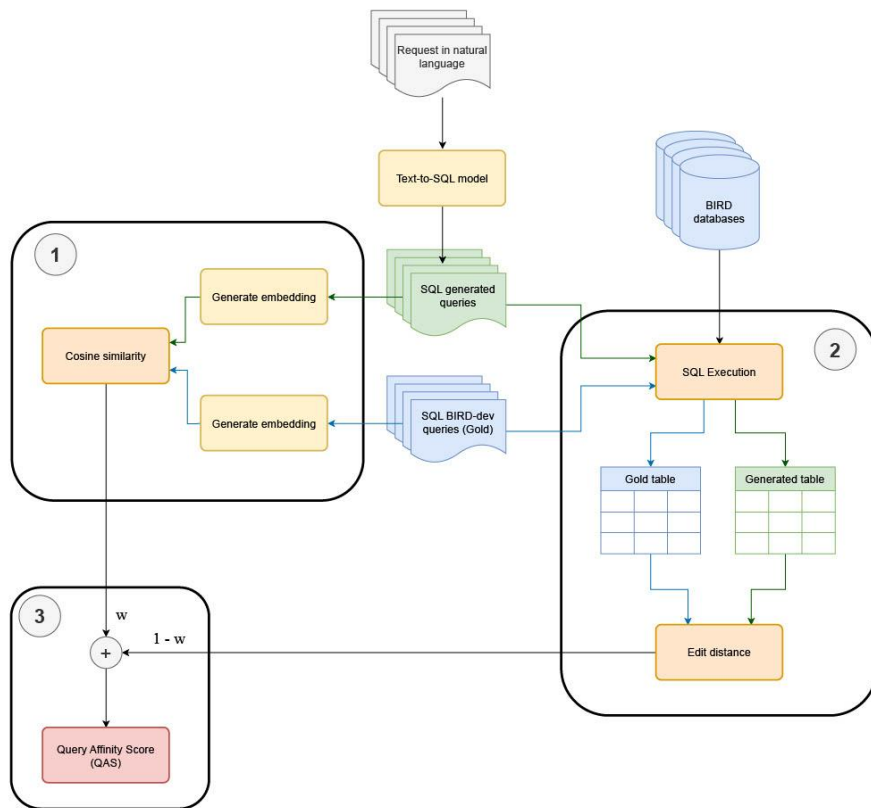
EX = 1



player_name
Lionel Messi
Lionel Messi
Lionel Messi
Lionel Messi
Lionel Messi
Lionel Messi
Lionel Messi

QAS (Pipeline)

1. **Compute semantic similarity.** (No longer limited by identical strings.)
2. **Compute table similarity.** (We do not use sets.)
3. **Query Affinity Score (QAS):** linear combination of the two similarities.



QAS (Data)

- **BIRD benchmark [22] is designed to be realistic and complex.**
- New state of the art for the text-to-SQL task.
- 12k examples.
- 37 professional domains.
- 95 different databases.



We used the DEV branch.

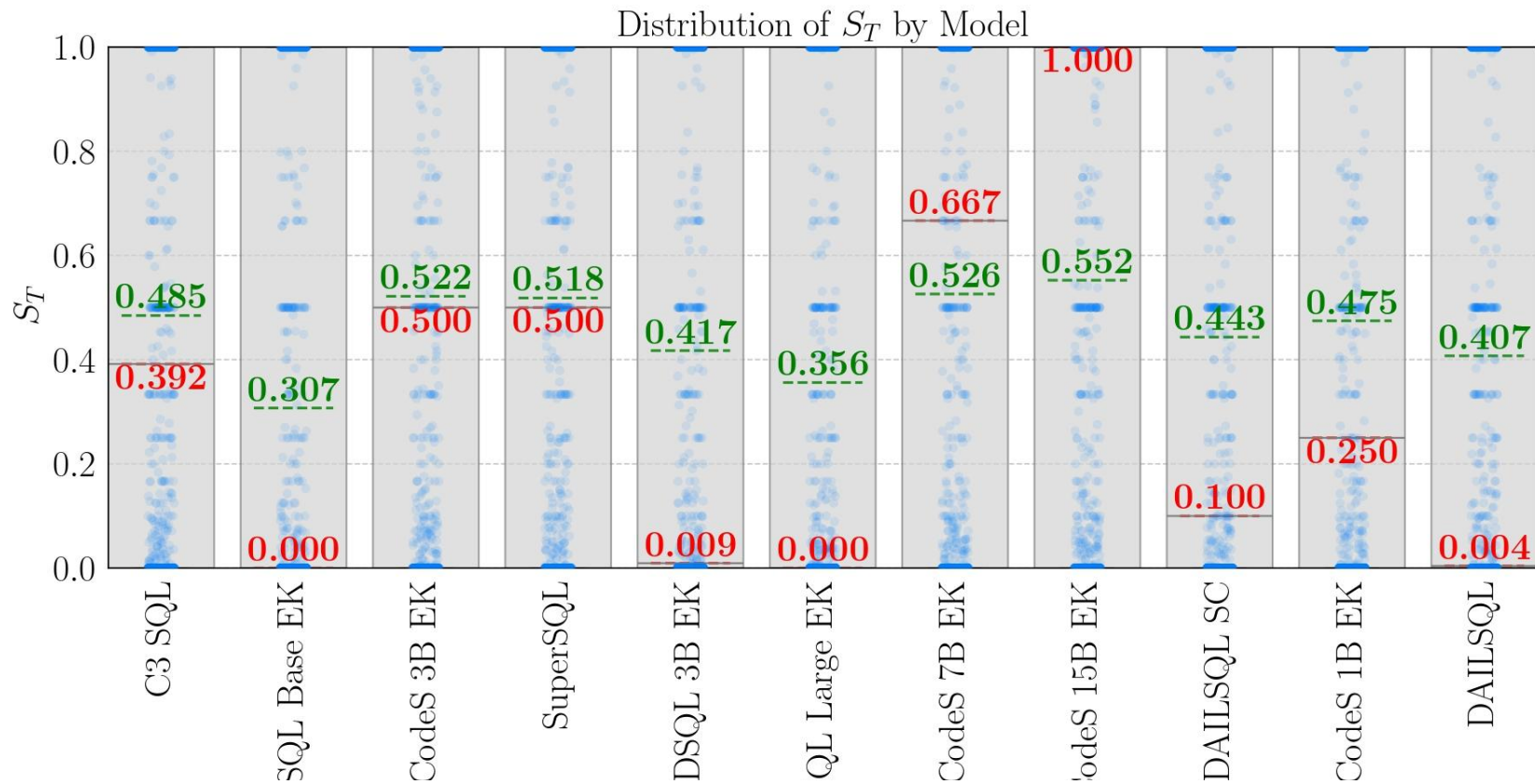
Query Affinity Score (QAS)

How many changes must be made to transform one table into the other (normalized). If 1, then everything needs to change.

$$S_T = 1 - \frac{1}{\max(n, m)} \sum_{i=1}^n \frac{\min_{j=1}^m d(c_i, b_j)}{\max(\text{len}(c_1), \text{len}(b_1))}, \quad \forall i = 1, 2, \dots, n \quad \text{and} \quad \forall j = 1, 2, \dots, m$$

$$\text{QAS} = w \cdot S_C + (1 - w) \cdot S_T$$

QAS (S_T Distribution)



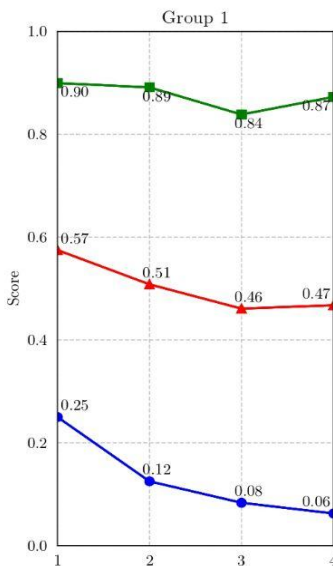
QAS (Make sense)

Query GOLD

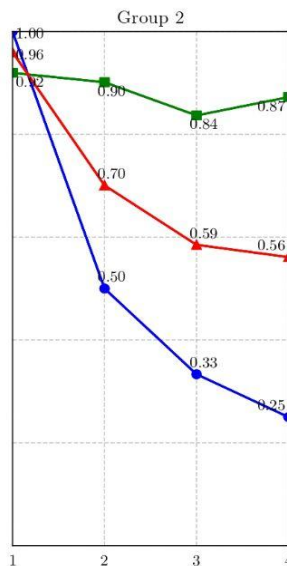
```
SELECT T2.MailStreet  
FROM frpm AS T1 INNER JOIN schools AS T2 ON  
T1.CDSCode = T2.CDSCode  
ORDER BY T1.`FRPM Count (K-12)` DESC  
LIMIT 1;
```

QAS (Make sense)

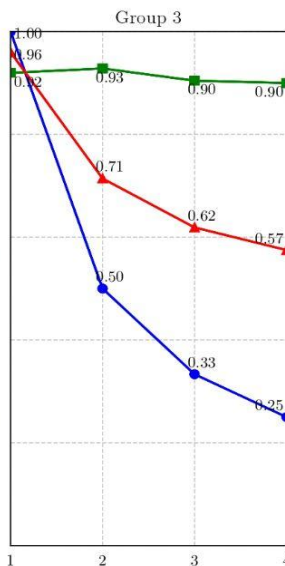
Added 3 columns and progressive increase in rows



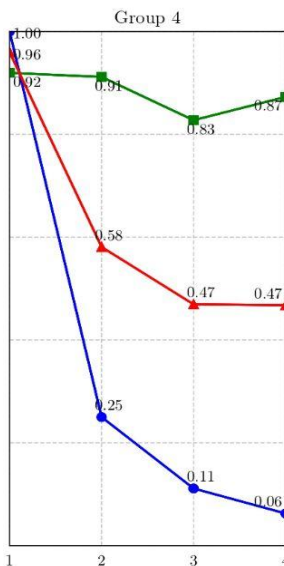
Progressive addition of rows



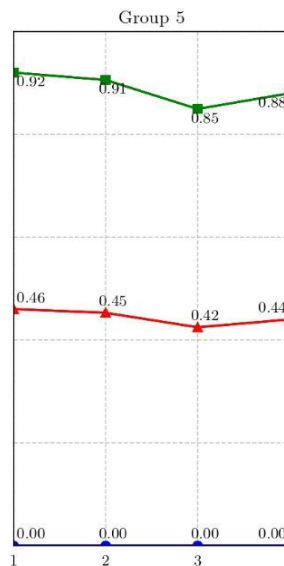
Progressive addition of columns



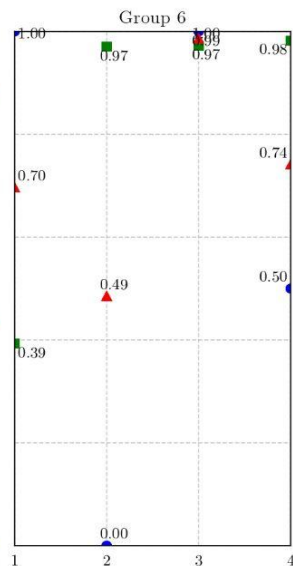
Progressive addition of pairs of rows and columns



Added progressive pairs of rows and columns, changed ordering to ASC



Queries representing edge cases



S_T S_C QAS

Casi limite 1/4

Query GOLD

```
SELECT T2.MailStreet
FROM frpm AS T1 INNER JOIN schools AS T2 ON
T1.CDSCode = T2.CDSCode
ORDER BY T1.`FRPM Count (K-12)` DESC
LIMIT 1;
```

MailStreet

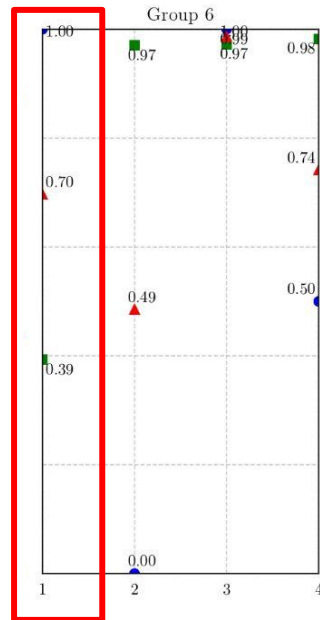
14429 South Downey Avenue

Query 1

```
SELECT MailStreet
FROM schools
WHERE CDSCode == 19648731936749;
```

MailStreet

14429 South Downey Avenue



Casi limite 2/4

Query GOLD

```
SELECT T2.MailStreet
FROM frpm AS T1 INNER JOIN schools AS T2 ON
T1.CDSCode = T2.CDSCode
ORDER BY T1.`FRPM Count (K-12)` DESC
LIMIT 1;
```

MailStreet

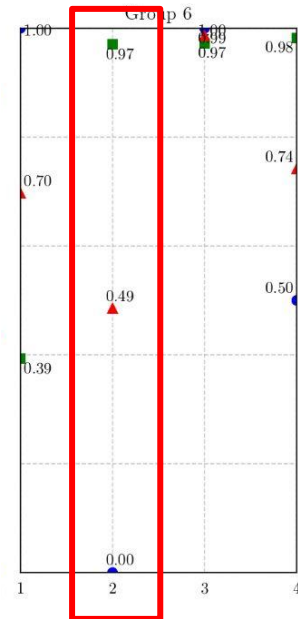
14429 South Downey Avenue

Query 2

```
SELECT T2.mailstreet
FROM satscores AS T1 INNER JOIN schools AS T2 ON
T1.cds = T2.CDSCode
WHERE T2.cdscode IN (
  SELECT cdscode
  FROM frpm
  ORDER BY "frpm count (k-12)" ASC
  LIMIT 1);
```

MailStreet

None



Casi limite 3/4

Query GOLD

```
SELECT T2.MailStreet
FROM frpm AS T1 INNER JOIN schools AS T2 ON
T1.CDSCode = T2.CDSCode
ORDER BY T1.`FRPM Count (K-12)` DESC
LIMIT 1;
```

MailStreet

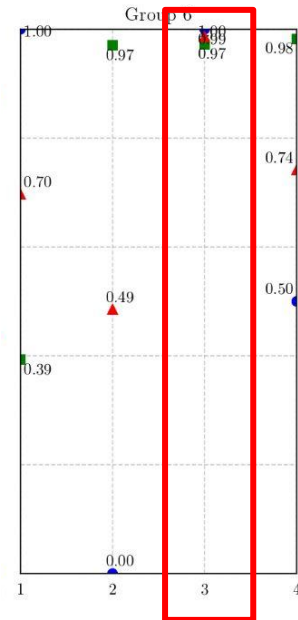
14429 South Downey Avenue

Query 3

```
SELECT T2.mailstreet
FROM satscores AS T1 INNER JOIN schools AS T2 ON
T1.cds = T2.CDSCode
WHERE T2.cdscode IN (
  SELECT cdscode
  FROM frpm
  ORDER BY "frpm count (k-12)" DESC
  LIMIT 1);
```

MailStreet

14429 South Downey Avenue



Casi limite 4/4

Query GOLD

```
SELECT T2.MailStreet
FROM frpm AS T1 INNER JOIN schools AS T2 ON
T1.CDSCode = T2.CDSCode
ORDER BY T1.`FRPM Count (K-12)` DESC
LIMIT 1;
```

MailStreet

14429 South Downey Avenue

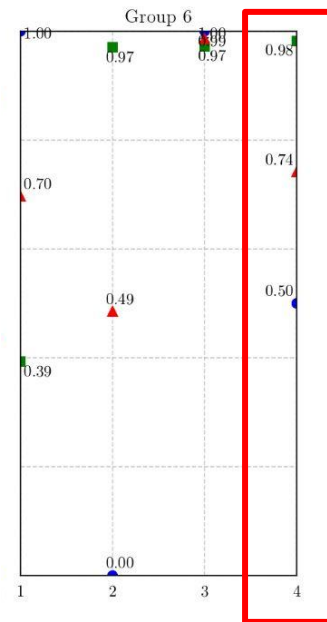
Query 4

```
SELECT T2.MailStreet
FROM frpm AS T1 INNER JOIN schools AS T2 ON
T1.CDSCode = T2.CDSCode
ORDER BY T1.`FRPM Count (K-12)` DESC
LIMIT 2;
```

MailStreet

14429 South Downey Avenue

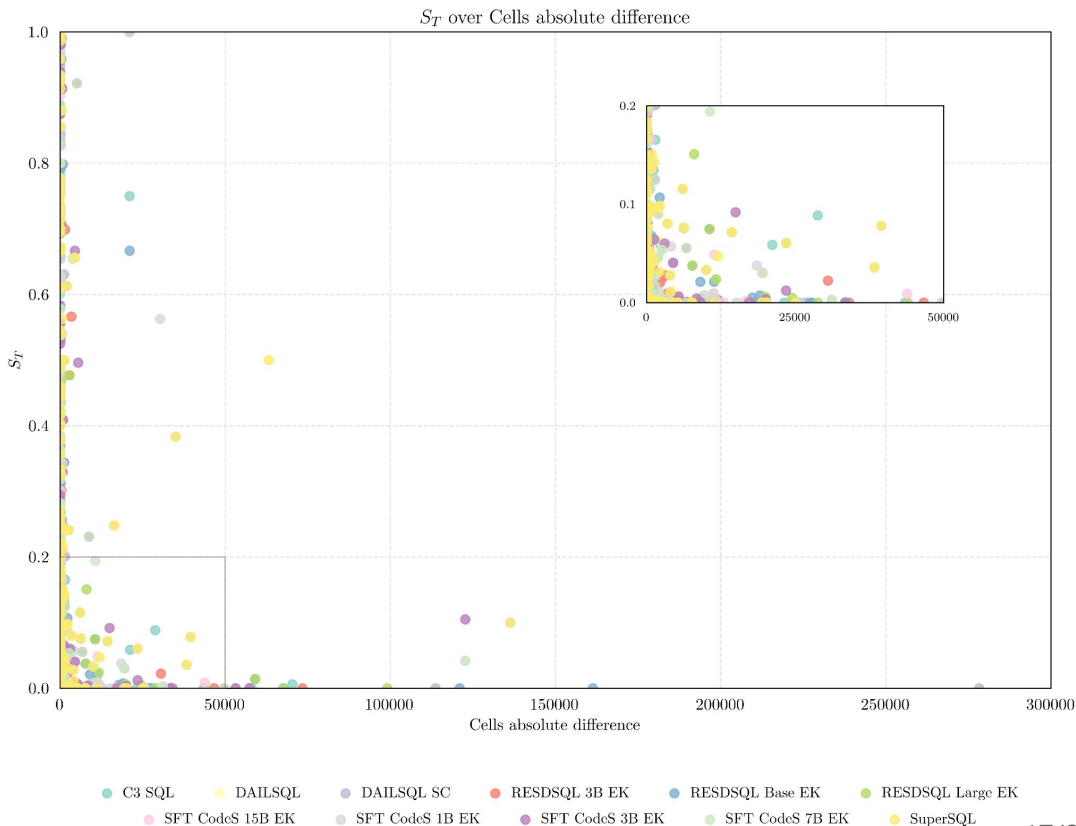
PO Box 792



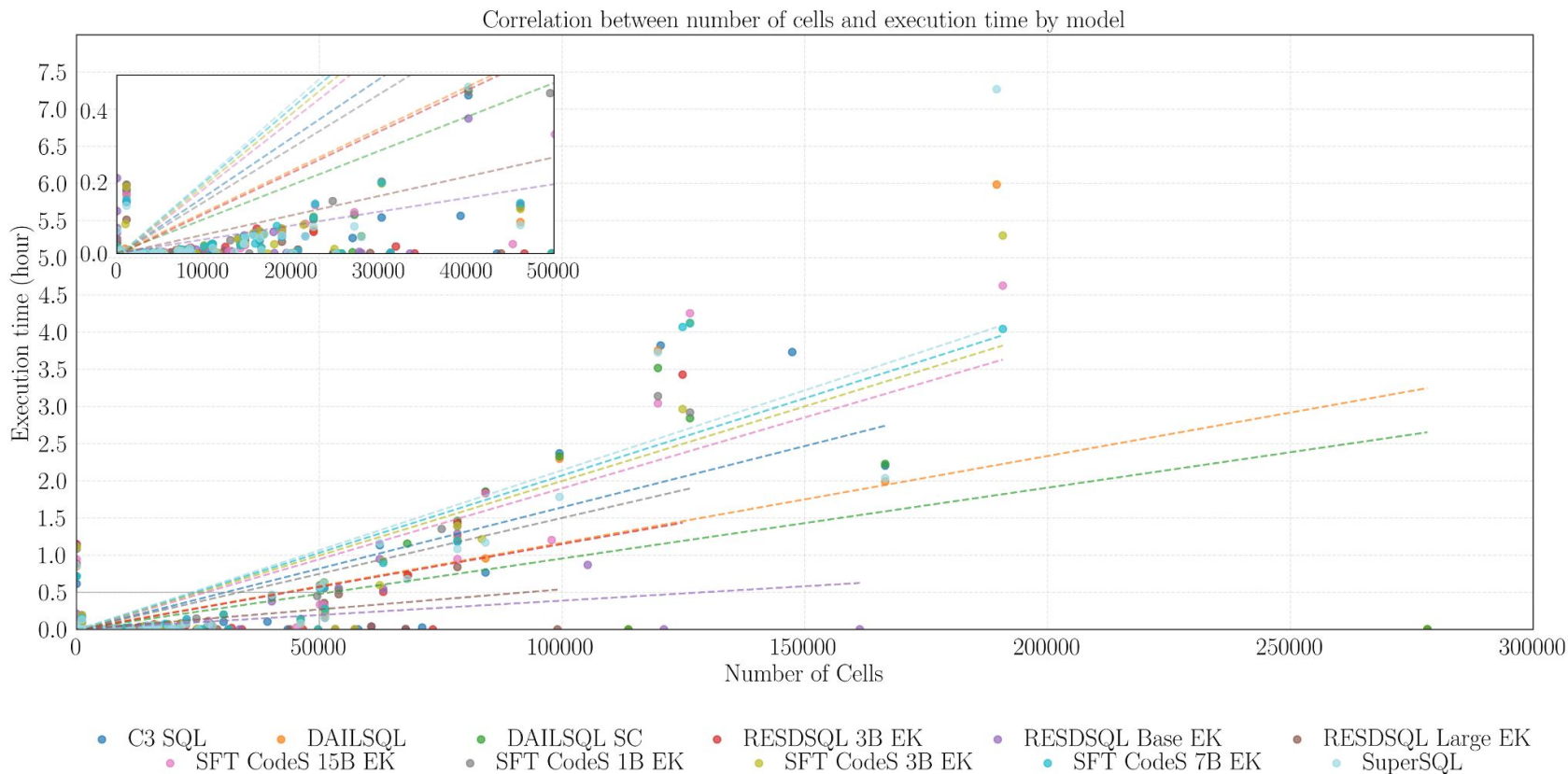
Other Simple Solutions Do Not Work

Correlation between S_T and the difference in the number of table cells.

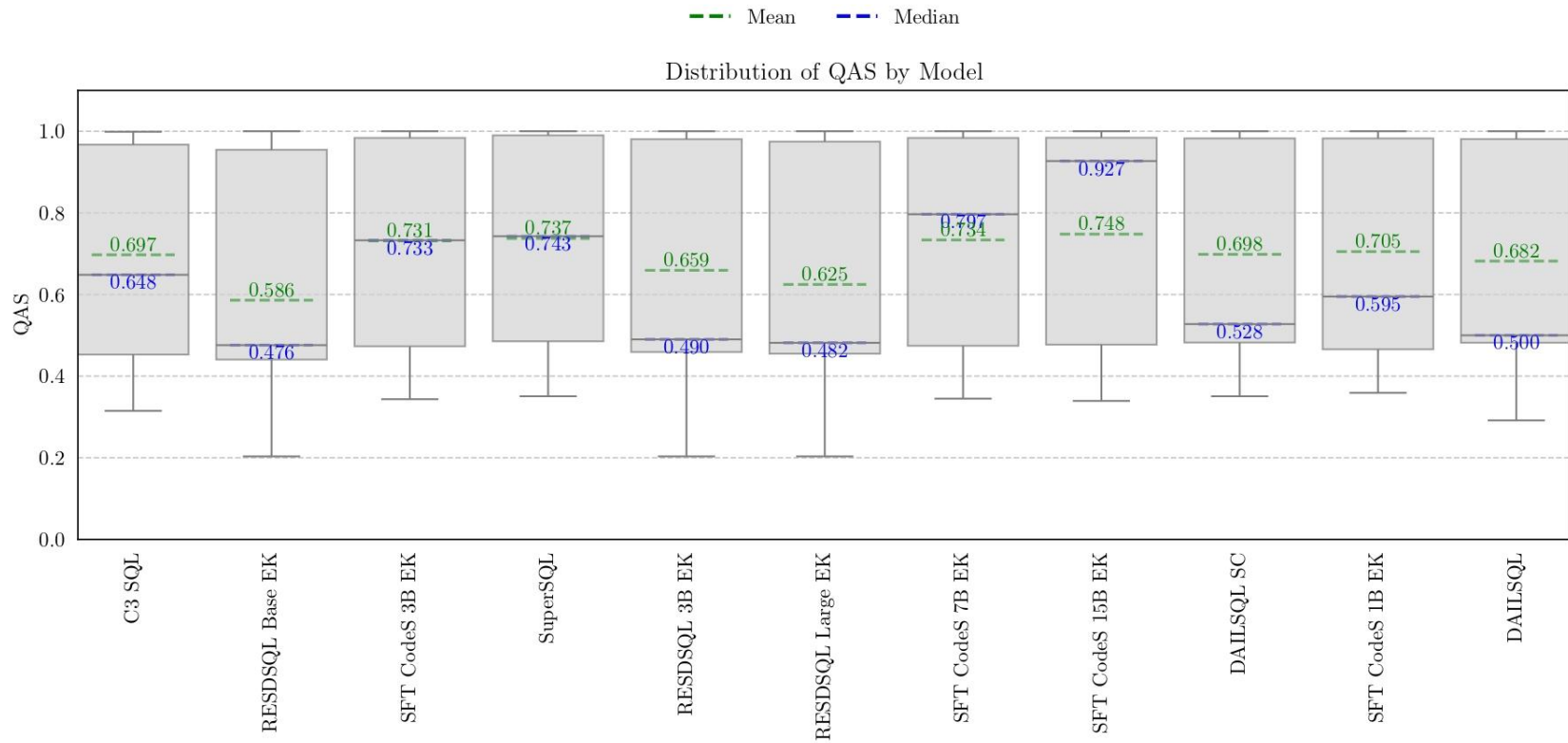
There is no correlation between the two measures; therefore, the difference in the number of cells cannot substitute for the S_T measure.



QAS (Time)



QAS (Distributions)



QAS (Conclusion)

- More detailed similarity enables **more effective evaluation of generated queries (allows more precise comparisons between models)**.
- Ability to provide more precise feedback to the model during the training phase.
- **QAS considers both the semantics of SQL queries and the results of their execution.**
- QAS is a linear combination of S_T and S_C , which allows adjusting the importance and contribution of these two components.

III

Sustainability and Green AI

Green feature selection (HotCat)

L. De la Cal, Y. Cao, A. Irmak Ercevik, **G. Pinna**, L. Twist, D. Williams, K. Even-Mendoza, W. B. Langdon, H. D. Menendez, F. Sarro, "HotCat: Green and Effective Feature Selection toward Hotfix Bug Taxonomy," *17th Symposium on Search-Based Software Engineering (SSBSE)*, Springer, 2025.

HotCat (Introduction)

Software maintenance represents a critical phase in the software development lifecycle, consuming a substantial portion of overall development resources.

Within this domain, hotfixes constitute a particularly challenging category of software patches that address urgent issues requiring immediate deployment to production systems [26].

HotCat (Introduction)

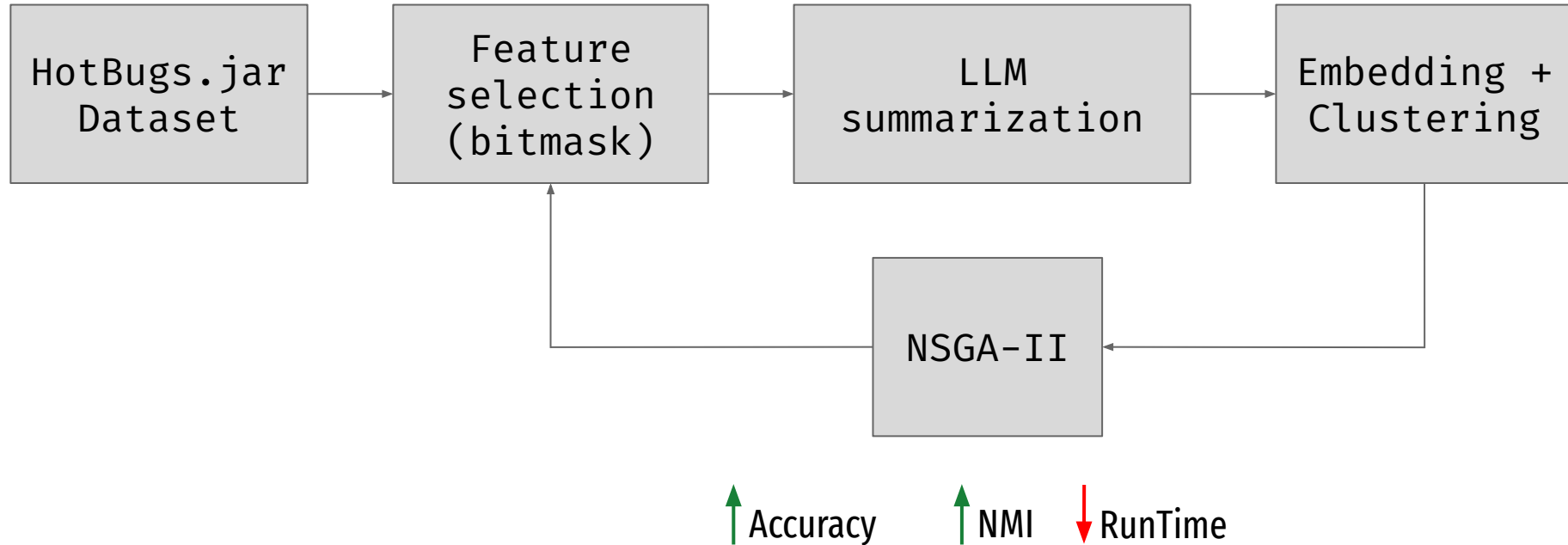
Unlike regular maintenance patches that follow standard release cycles, hotfixes must balance the urgency of rapid deployment with the fundamental requirement of system stability, creating inherent tensions between time criticality and code quality assurance [25].

The systematic classification and organization of bugs into coherent taxonomies has long been recognized as essential for effective software maintenance [26]. A well-defined bug taxonomy enables development teams to prioritize resources, identify recurring patterns, and implement targeted preventive measures.

HotCat (For Practitioner)

- **Resource prioritization:** different categories (crash, security, test) require different protocols.
- **Recurring patterns:** identify architectural weaknesses and areas for preventive investment.
- **Automatic triage:** automatic routing of bug reports to appropriate teams.

HotCat (Pipeline)



HotCat (experimental)

Used the **HotBugs.jar benchmark [25]**.

- 88 real hot bugs (augmentation to 155 for categories balance).
- Augmented with metadata (e.g., code diff, temporal metrics) total **18 features**
- **17 Bugs category** (good real coverage)

LLM for summarization is **Mistral** (Le Chat).

For embedding we used Sentence-BERT [12], for clustering K-means.

Validation with new 51 hot bugs annotated by human.

HotCat (experimental)

NSGA-II [11][34]

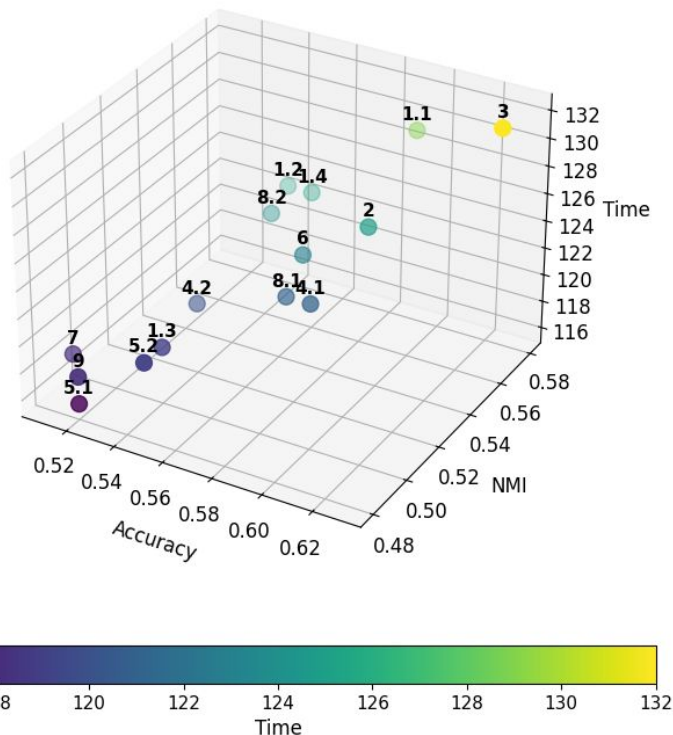
- 20 individuals
- 20 generations
- Binary Crossover
- Bit-flip mutation
- Maximize Accuracy, Maximize NMI, Minimize Runtime

The **bitmask** is an **18-bit binary vector** (one bit per feature) where 1 means "include this feature" and 0 means "exclude it," serving as the individual that NSGA-II evolves to find the optimal feature combination.

$2^{18} = 262,144$ possible configurations

HotCat (Results)

3D Pareto Front Visualization



Individual	Accuracy (%)	NMI	Seconds
1.1	59	0.58	129
1.2	55	0.56	125
1.3	54	0.50	119
1.4	56	0.56	125
2	59	0.55	125
3	63	0.57	132
4.1	58	0.53	121
4.2	54	0.52	120
5.1	52	0.48	116
5.2	54	0.49	119
6	57	0.54	123
7	51	0.49	118
8.1	57	0.53	121
8.2	55	0.55	124
9	52	0.48	118

HotCat (Conclusion)

HotCat demonstrates that classification accuracy and computational efficiency can be jointly optimized through NSGA-II (Green AI principles [30]).

Maximum accuracy (63%) requires only 3 additional seconds compared to the balanced configuration (132s vs. 129s).

Feature selection serves a dual purpose:

- it reduces LLM inference costs (shorter prompts)
- improves classification quality by removing noisy features.

Greener Agent for Greener Code (GA4GC)

J. Gong, Y. Bian, L. De la Cal, **G. Pinna**, A. Uteem, D. Williams, M. Zamorano, K. Even-Mendoza, W. B. Langdon, H. D. Menendez, F. Sarro, "GA4GC: Greener Agent for Greener Code via Multi-Objective Configuration Optimization," *17th Symposium on Search-Based Software Engineering (SSBSE)*, Springer, 2025.

GA4GC (The problem)

- **Task** LLM-powered **coding agents** are powerful for complex, real-world **code optimization** tasks.
- **Cost** a single run on a SWE-Bench problem can consume over **100,000 tokens** [1].
- **Issue** the energy cost to run the agent can be a "**net energy loss**", requiring hundreds of thousands of code executions to see a benefit [2].
- **Challenge** this high cost creates **scalability** challenges and conflicts with **Green SE** principles.

GA4GC (Pipeline)

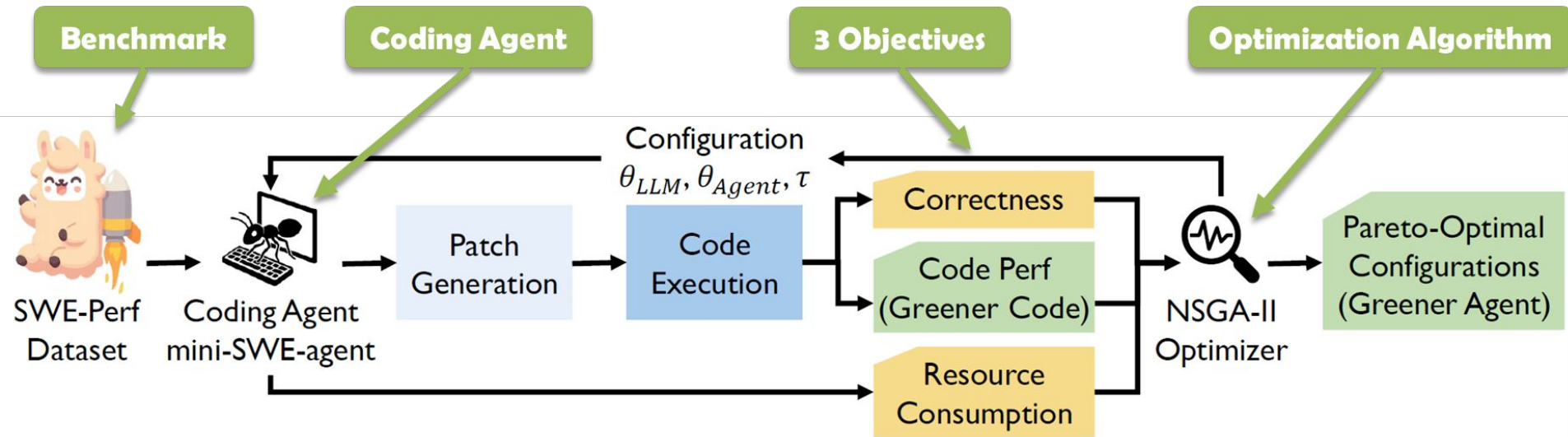


Fig. 1. GA4GC workflow of multi-objective configuration optimization.

GA4GC (experimental)

Used the **SWE-Perf benchmark** [28].

- Benchmark for evaluating code performance optimization capabilities in real repositories.
- **Training set:** 9 instances
- **Validation set:** 3 instances

LLM Gemini 2.5 Pro (API) [29]

Server Google Cloud Platform with 4 CPUs, 16GB RAM

GA4GC (experimental)

NSGA-II

- 5 individuals
- 5 generations
- Binary Crossover
- Polynomial mutation
- Maximize Correctness, Maximize Performance, Minimize Runtime

We trained **3 Random Forest regressors** (one for each objective: correctness, performance, runtime).

Parameters: n_estimators=100; max_depth=5

GA4GC (AI Agent Params to Optimize)

Category	Hyperparameter	Range/Values	Type	Description
LLM	Temperature	[0.0, 1.0]	Continuous	Randomness in token selection
	Top_p	[0.1, 1.0]	Continuous	Vocabulary size limit
	Max_tokens	[512, 4096]	Integer	Maximum response length
Agent	Step_limit	[10, 40]	Integer	Max LLM calls per instance
	Cost_limit	[\$3.0, \$10.0]	Continuous	Total LLM usage budget
	Env_timeout	[40s, 60s]	Integer	Environment operation timeout
	LLM_timeout	[40s, 60s]	Integer	Individual LLM call timeout
Prompt	Template	{1, 2, 3}	Categorical	Template variant (5, 10, or 20 steps)

GA4GC (Results GA4GC vs. default)

Table 2. Default vs. GA4GC-optimized configurations. Corr=Correctness, Perf= performance gain (%), Runt=runtime (s), HV=hypervolume, VHV=validation hypervolume. See Table 1 for other definitions. Green cells indicate improvements over default.

Config	Temp	TopP	Token	Step	Cost	ETi	LTi	Pr	Corr	Perf	Runt	HV (%)	VHV (%)
Default	0.000	1.000	4096	240	3.00	60	60	-	2/9	0.00	1513.3	0.52	1.1
#4	0.085	0.135	1120	36	9.26	41	57	2	4/9	0.00	943.1	5.82	4.1
#5	0.692	0.384	2972	38	6.73	40	56	3	8/9	6.43	984.8	70.28	14.9
#9	0.725	0.412	2972	22	6.73	43	41	3	7/9	0.00	958.1	9.25	21.6
#15	0.657	0.384	2972	38	6.73	40	56	2	7/9	10.67	1400.1	33.42	2.7
#16	0.085	0.131	1120	36	6.91	41	57	2	0/9	0.00	853.3	1.10	21.6

GA4GC (Results GA4GC vs. default)

Table 2. Default vs. GA4GC-optimized configurations. Corr=Correctness, Perf= performance gain (%), Runt=runtime (s), HV=hypervolume, VHV=validation hypervolume. See Table 1 for other definitions. Green cells indicate improvements over default.

Config	Temp	TopP	Token	Step	Cost	ETi	LTi	Pr	Corr	Perf	Runt	HV (%)	VHV (%)
Default	0.000	1.000	4096	240	3.00	60	60	-	2/9	0.00	1513.3	0.52	1.1
#4	0.085	0.135	1120	36	9.26	41	57	2	4/9	0.00	943.1	5.82	4.1
#5	0.692	0.384	2972	38	6.73	40	56	3	8/9	6.43	984.8	70.28	14.9
#9	0.725	0.412	2972	22	6.73	43	41	3	7/9	0.00	958.1	9.25	21.6
#15	0.657	0.384	2972	38	6.73	40	56	2	7/9	10.67	1400.1	33.42	2.7
#16	0.085	0.131	1120	36	6.91	41	57	2	0/9	0.00	853.3	1.10	21.6

GA4GC achieves **37.7% faster** runtime while improving correctness, **135 times** higher hypervolume, and **4/5** Pareto front configurations dominating the default while all maintaining superior hypervolume on **unseen tasks**.

GA4GC (Results Influence Analysis)

Table 3. Random Forest feature importance for hyperparameters on optimization objectives. Colors indicate importance: Low (0.0-0.1), Medium (0.1-0.2), High (>0.2).

Category	Hyperparameter	Correctness Impact	Performance Impact	Runtime Impact
LLM	Temperature	0.152	0.392	0.199
	Top_p	0.199	0.051	0.097
	Max_tokens	0.057	0.090	0.089
Agent	Step_limit	0.140	0.119	0.049
	Cost_limit	0.199	0.076	0.128
	Env_timeout	0.060	0.034	0.298
	LLM_timeout	0.120	0.109	0.102
Prompt	Template Variant	0.072	0.130	0.038

GA4GC (Results Influence Analysis)

Table 3. Random Forest feature importance for hyperparameters on optimization objectives. Colors indicate importance: Low (0.0-0.1), Medium (0.1-0.2), High (>0.2).

Category		Hyperparameter	Correctness Impact	Performance Impact	Runtime Impact
LLM		Temperature	0.152	0.392	0.199
		Top_p	0.199	0.051	0.097
		Max_tokens	0.057	0.090	0.089
Agent		Step_limit	0.140	0.119	0.049
		Cost_limit	0.199	0.076	0.128
		Env_timeout	0.060	0.034	0.298
		LLM_timeout	0.120	0.109	0.102
Prompt		Template Variant	0.072	0.130	0.038

Temperature shows **highest overall influence**, LLM hyperparameters primarily impact task effectiveness while agent constraints affect **resource consumption**, confirming the need for GA4GC in green agent deployment.

GA4GC (Results Actionable Strategies)

1. **Runtime-Critical Scenarios** Use low temperature (0–0.1) restrictive top_p (0.13–0.14) to **minimize exploration** overhead. Use moderate max_tokens (1120–2000) and step limits (20–36).
2. **Performance-Critical Scenarios** Use moderate temperature (0.65–0.70) with balanced top_p (0.38–0.41) to enable **creative optimization** strategies. Use higher cost budgets (\$6.5–\$9.5) and prompt templates optimized for performance tasks.
3. **Context-Specific Scenarios** For practitioners with specific requirements, we recommend **running GA4GC** to discover the most accurate Pareto-optimal configurations tailored to their deployment priorities.

GA4GC (Conclusion)

- Introduce **GA4GC**, a framework to optimize the agent-code performance trade-off.
- Achieve **135-fold** hypervolume improvement over default.
- Identify **temperature** as the most critical parameter.
- Provide **actionable suggestions** for coding agent practitioners across three scenarios.

IV

Conclusion

Limitation

Legal domain:

- Focus on judgments of the Italian Constitutional Court.
- The pool of 75 participants might not represent the full diversity of the population.

Code generation and GI:

- The PSB2 benchmark represents a specific class of algorithmic problems.
- The versions of the evaluated models are specific, and current performance may differ.

Limitation

Text-to-SQL:

- QAS has high computational costs for very large result tables, due to the linear relationship between computation time and number of cells.

Sustainability:

- Limited NSGA-II evaluation budget;
- modest dataset in HotCat;
- runtime as a proxy rather than direct measurement of energy and emissions.

Answers RQ1

How can we develop more accurate and meaningful evaluation methodologies for LLM outputs that capture semantic correctness and practical utility beyond superficial metrics?

Meaningful LLM evaluation requires domain-specific approaches that capture genuine utility, not superficial similarity.

Answers RQ2

To what extent can LLMs democratize access to specialized knowledge in the legal and software engineering domains, and what are the limitations and risks of current systems in these contexts?

Significant potential, but systematic limitations and critical risks prevent reliable deployment without safeguards.

Risk of misinformation and misleading content; Risk of excessive dependence and displacement of expertise; Risk of subtle error propagation; Risk of calibration misalignment.

Answers RQ3

What post-processing and hybrid approaches can improve the reliability of LLM outputs in high-stakes applications?

Genetic Improvement with dynamically specialized grammars significantly improves LLM code on 25 problems and 5 models. Self-correction is ineffective — a fundamentally different search mechanism is needed.

Answers RQ4

How can we develop and deploy LLM-based systems while minimizing their environmental footprint and ensuring sustainable resource usage?

Computational efficiency and task effectiveness are jointly optimizable through MOO. Configuration optimization is an underexplored sustainability lever, without the need for retraining.

Conclusions

Three key lessons emerge from this work:

1. **Accessibility-Accuracy Paradox.** The fluency that makes LLMs useful for democratization also makes their errors harder to detect.
2. **Hybridization over Isolation.** Neither LLMs alone nor traditional methods are sufficient. The hybrid paradigm achieves what no approach achieves in isolation.
3. **Sustainability as Prerequisite.** Sustainability is not merely a desirable property but a prerequisite for responsible large-scale deployment.

References

- [1] Chen, M. et al. "Evaluating Large Language Models Trained on Code." arXiv:2107.03374, 2021.
- [2] Coignon, T. "The Hidden Environmental Cost of Code Optimization Agents." 2025.
- [3] Pinna, G. et al. "Enhancing Large Language Models-Based Code Generation by Leveraging Genetic Improvement." EuroGP, 2024.
- [4] Helmuth, T. and Kelly, P. "PSB2: The Second Program Synthesis Benchmark Suite." GECCO, 2021.
- [5] Pinna, G. et al. "Exploring the Effect of Genetic Improvement for Large Language Models-Generated Code." SN Computer Science, 2025.
- [6] OpenAI. "ChatGPT." <https://openai.com/chatgpt>, 2022.
- [7] Achiam, J. et al. "GPT-4 Technical Report." arXiv:2303.08774, 2023.
- [8] Touvron, H. et al. "LLaMA 2: Open Foundation and Fine-Tuned Chat Models." arXiv:2307.09288, 2023.
- [9] Taori, R. et al. "Alpaca: A Strong, Replicable Instruction-Following Model." Stanford, 2023.
- [10] Vaswani, A. et al. "Attention Is All You Need." NeurIPS, 2017.
- [11] Deb, K. et al. "A Fast and Elitist Multiobjective Genetic Algorithm: NSGA-II." IEEE Trans. Evol. Comp., 2002.
- [12] Reimers, N. and Gurevych, I. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." EMNLP, 2019.
- [13] Bommasani, R. et al. "On the Opportunities and Risks of Foundation Models." arXiv:2108.07258, 2021.
- [14] Chang, Y. et al. "A Survey on Evaluation of Large Language Models." ACM TIST, 2024.
- [15] Zodi, Z. "The Limits of Plain Legal Language." 2019.
- [16] Kore, R. et al. "Legal Document Summarization using NLP and ML Techniques." 2020.

References

- [17] Deroy, A. et al. "How Ready Are Pre-Trained Abstractive Models and LLMs for Legal Case Judgement Summarization?" 2023.
- [18] Achena, F. et al. "Legal Language Complexity and Access to Justice." 2023.
- [19] Pinna, G. et al. "From Courts to Comprehension: Can LLMs Make Judgments More Accessible?" WI-IAT, 2024.
- [20] Corte Costituzionale Italiana. Open Data – Sentenze e Massime. <https://www.cortecostituzionale.it>
- [21] Yu, T. et al. "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task." EMNLP, 2018.
- [22] Li, J. et al. "Can LLM Already Serve as a Database Interface? A Big Bench for Large-Scale Database Grounded Text-to-SQLs (BIRD)." NeurIPS, 2024.
- [23] Pinna, G. et al. "Redefining Text-to-SQL Metrics by Incorporating Semantic and Structural Similarity." 2025.
- [24] Islam, M. et al. "Runtime Bug Detection and Hotfix Deployment in Production Systems." 2023.
- [25] Hanna, M. et al. "HotBugs: A Dataset of Time-Critical Software Patches." 2025.
- [26] Chillarege, R. et al. "Orthogonal Defect Classification—A Concept for In-Process Measurements." IEEE TSE, 1992.
- [27] Even-Mendoza, K. et al. "PatchCat: Automatic Categorization of Software Patches." 2025.
- [28] He, Z. et al. "SWE-Perf: Benchmarking Code Performance Optimization via LLM Agents." 2025.
- [29] Comanici, G. et al. "Gemini 2.5: Pushing the Frontier with Advanced Reasoning." Google, 2025.
- [30] Schwartz, R. et al. "Green AI." Communications of the ACM, 2020.
- [31] Brown, T. et al. "Language Models are Few-Shot Learners." NeurIPS, 2020.
- [32] Anthropic. "Claude 3 Model Card." 2024

References

- [33] Touvron, H. et al. "LLaMA: Open and Efficient Foundation Language Models." arXiv:2302.13971, 2023.
- [34] Blank, J. and Deb, K. "pymoo: Multi-Objective Optimization in Python." IEEE Access, 2020.
- [35] GitHub. "mini-swe-agent." <https://github.com/princeton-nlp/SWE-agent>, 2024.
- [36] Zsolt Zodi. The limits of plain legal language: understanding the comprehensible style in law. International Journal of Law in Context, 15(3):246–262, 2019
- [37] Aniket Deroy, Kripabandhu Ghosh, and Saptarshi Ghosh. How ready are pre-trained abstractive models and llms for legal case judgement summarization? arXiv preprint arXiv:2306.01248, 2023.
- [38] Lauren Martin, Nick Whitehouse, Stephanie Yiu, Lizzie Catterson, and Rivindu Perera. Better call gpt, comparing large language models against lawyers. arXiv preprint arXiv:2401.16212, 2024.
- [39] Flavia Achena, David Preti, Davide Venditti, Leonardo Ranaldi, Cristina Giannone, Fabio Massimo Zanzotto, Andrea Favalli, Raniero Romagnoli, et al. Legal summarization: to each court its own model. 2023
- [40] Yupeng Chang, Xu Wang, Jindong Wang, Yuan Wu, Linyi Yang, Kaijie Zhu, Hao Chen, Xiaoyuan Yi, Cunxiang Wang, Yidong Wang, et al. A survey on evaluation of large language models. ACM Transactions on Intelligent Systems and Technology, 15(3):1–45, 2024.
- [41] [https://dati.cortecostituzionale.it/Scarica i dati/Scarica i dati](https://dati.cortecostituzionale.it/Scarica%20i%20dati/Scarica%20i%20dati)
- [42] <https://www.youtube.com/watch?v=3kF6Yr8EoaU>
- [43] Bahrini, A., et al.: ChatGPT: applications, opportunities, and threats. In: 2023 Systems and Information Engineering Design Symposium (SIEDS), pp. 274–279 (2023)
- [44] Sobania, D., Briesch, M., Rothlauf, F.: Choose your programming copilot: a comparison of the program synthesis performance of github copilot and genetic programming. In: Proceedings of the Genetic and Evolutionary Computation Conference, pp. 1019–1027 (2022)
- [45] Vaithilingam, P., Zhang, T., Glassman, E.L.: Expectation vs experience: evaluating the usability of code generation tools powered by large language models. In: Extended Abstracts of the 2022 CHI Conference on Human Factors in Computing Systems, CHI EA 2022. Association for Computing Machinery, New York (2022)

Thank you !

Thank you !

LLM

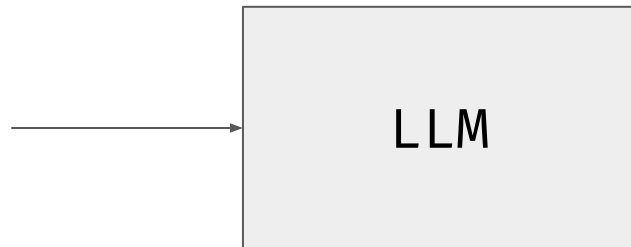
The cat is

LLM

The cat is

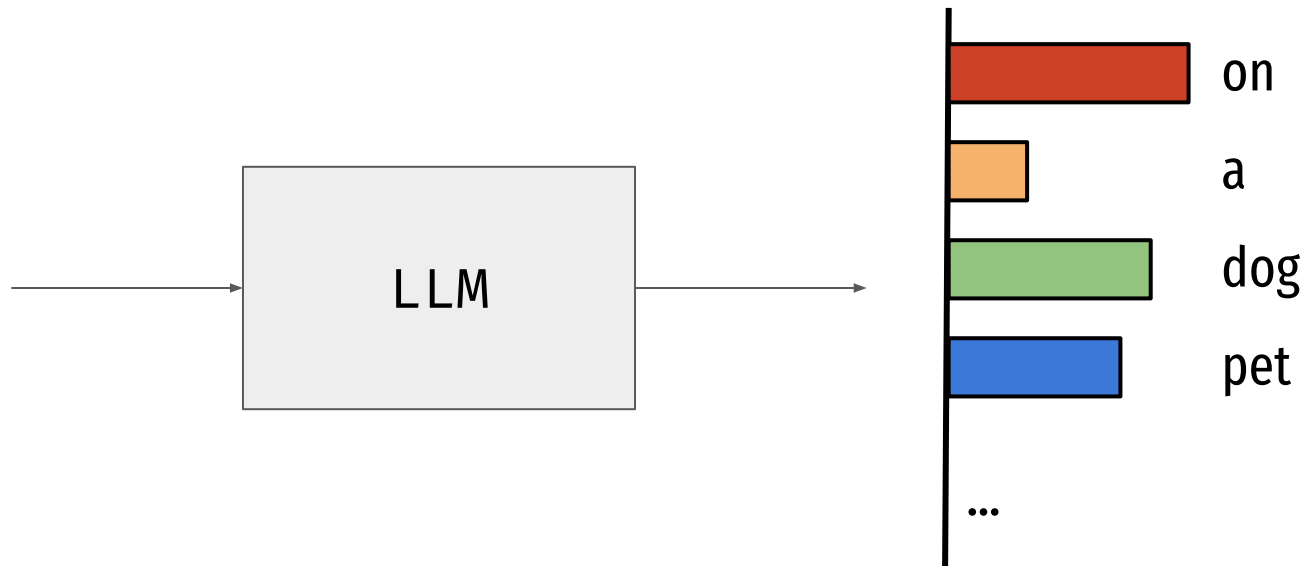
LLM

The cat is



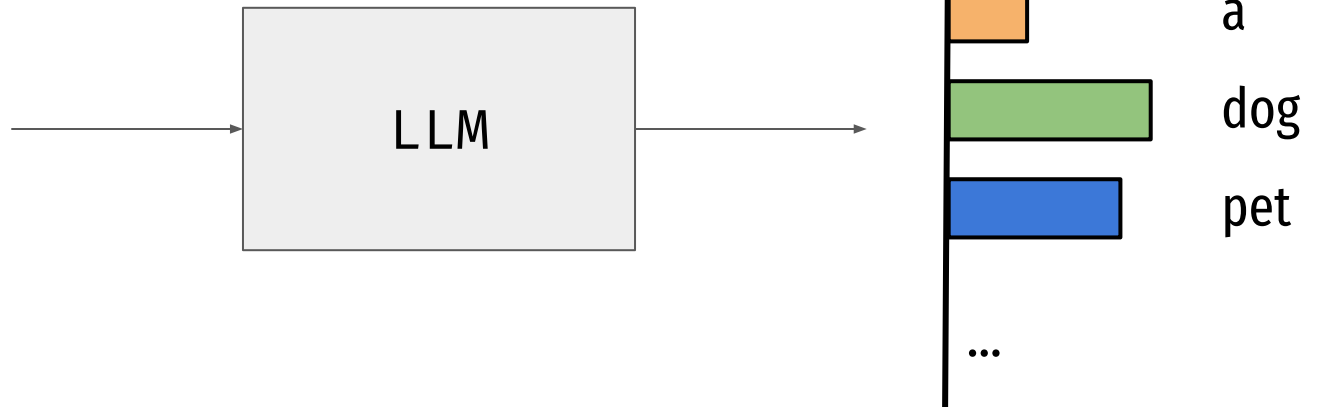
LLM

The cat is



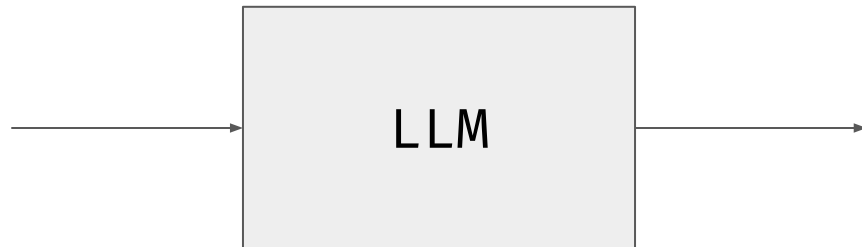
LLM

The cat is



LLM

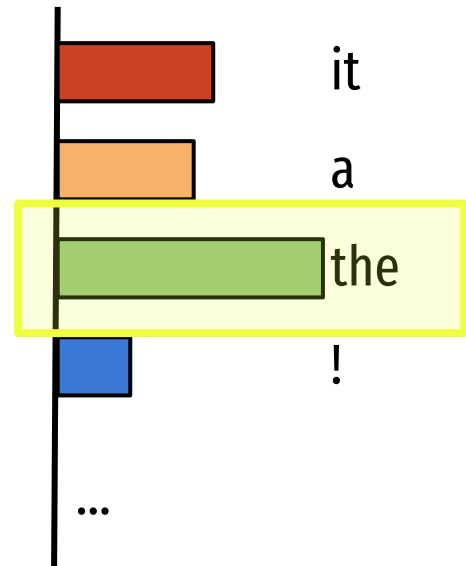
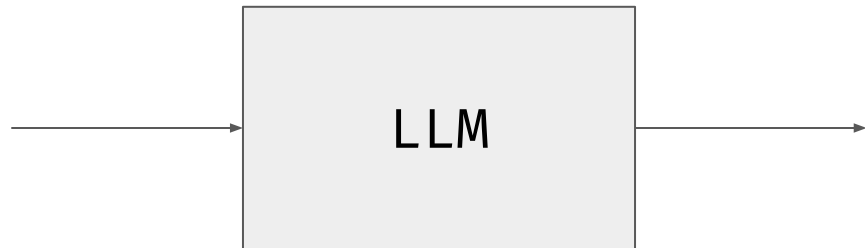
The cat is on



it
a
the
!
...

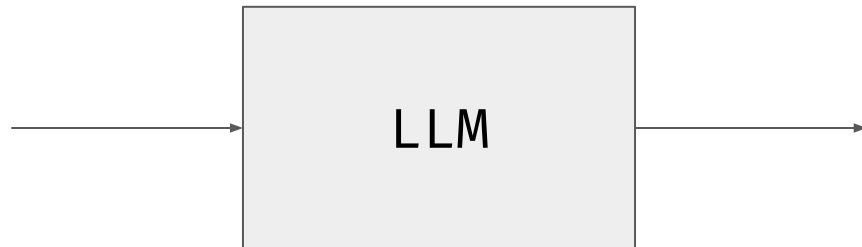
LLM

The cat is on



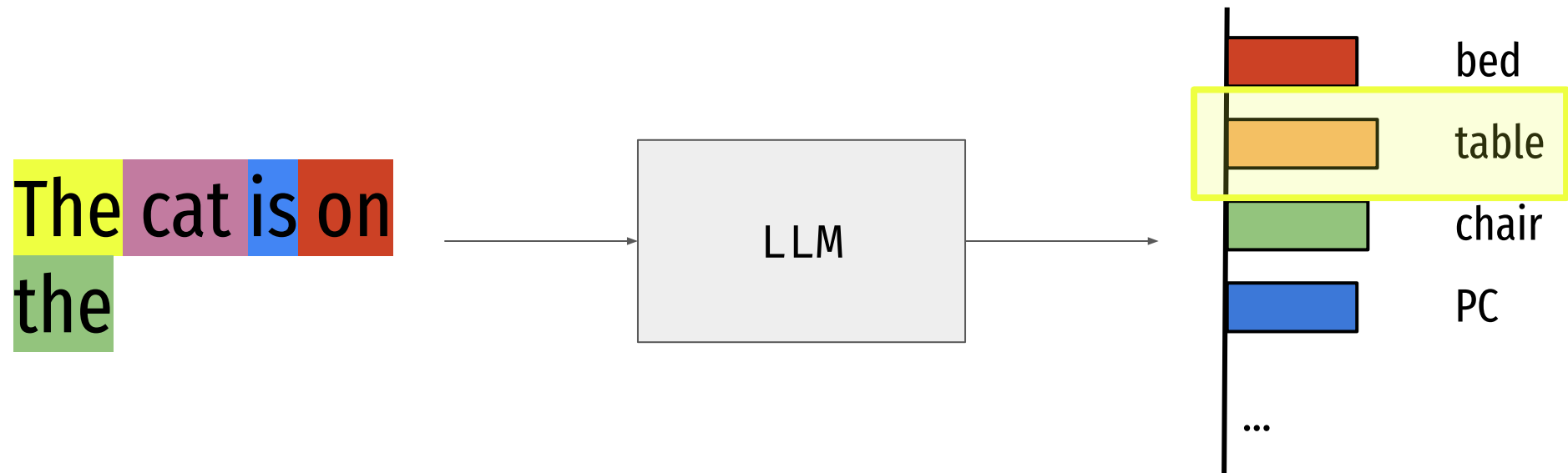
LLM

The cat is on
the



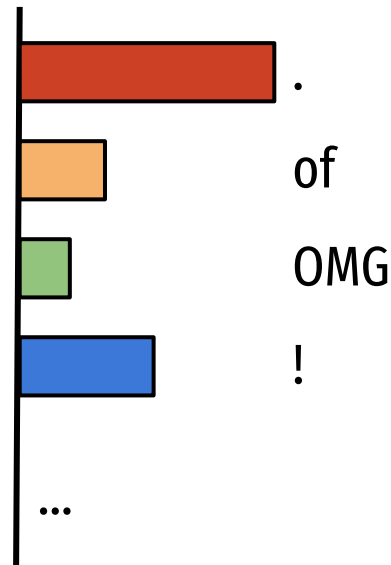
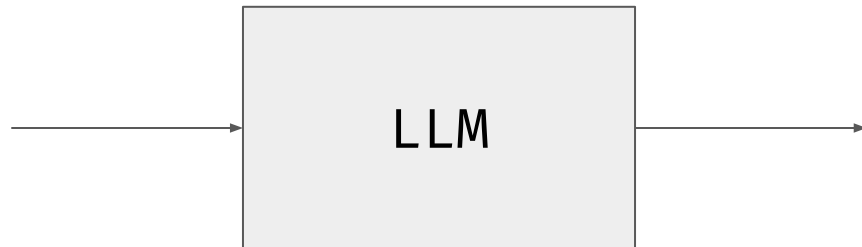
bed
table
chair
PC
...

LLM



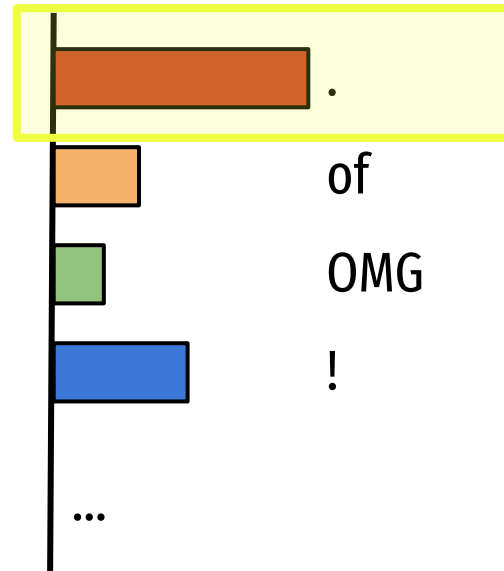
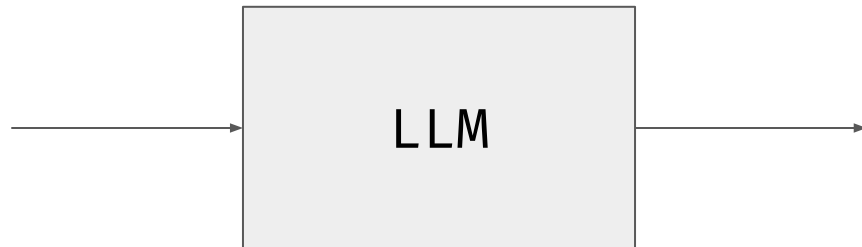
LLM

The cat is on
the table



LLM

The cat is on
the table



LLM

The cat is on
the table.

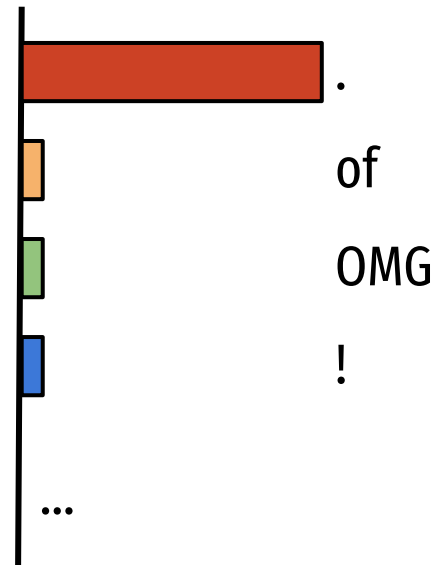
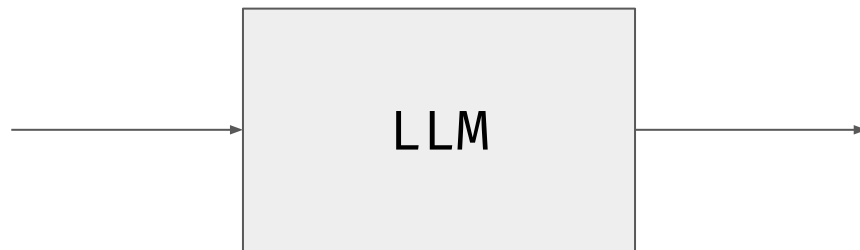
LLM (temperature)

LLM (Temperature) is a hyperparameter that controls the randomness of token selection during generation, modulating the probability distribution of the model's predictions: low temperatures make the output more deterministic, while high temperatures increase the variability and creativity of the responses.

LLM (temperature)

- **Low Temp** (ex. 0.1 - 0.3)

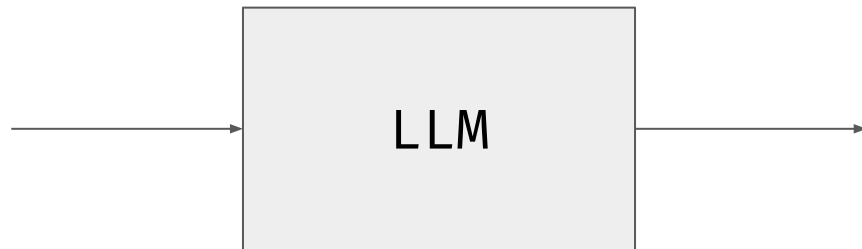
The cat is on
the table



LLM (temperature)

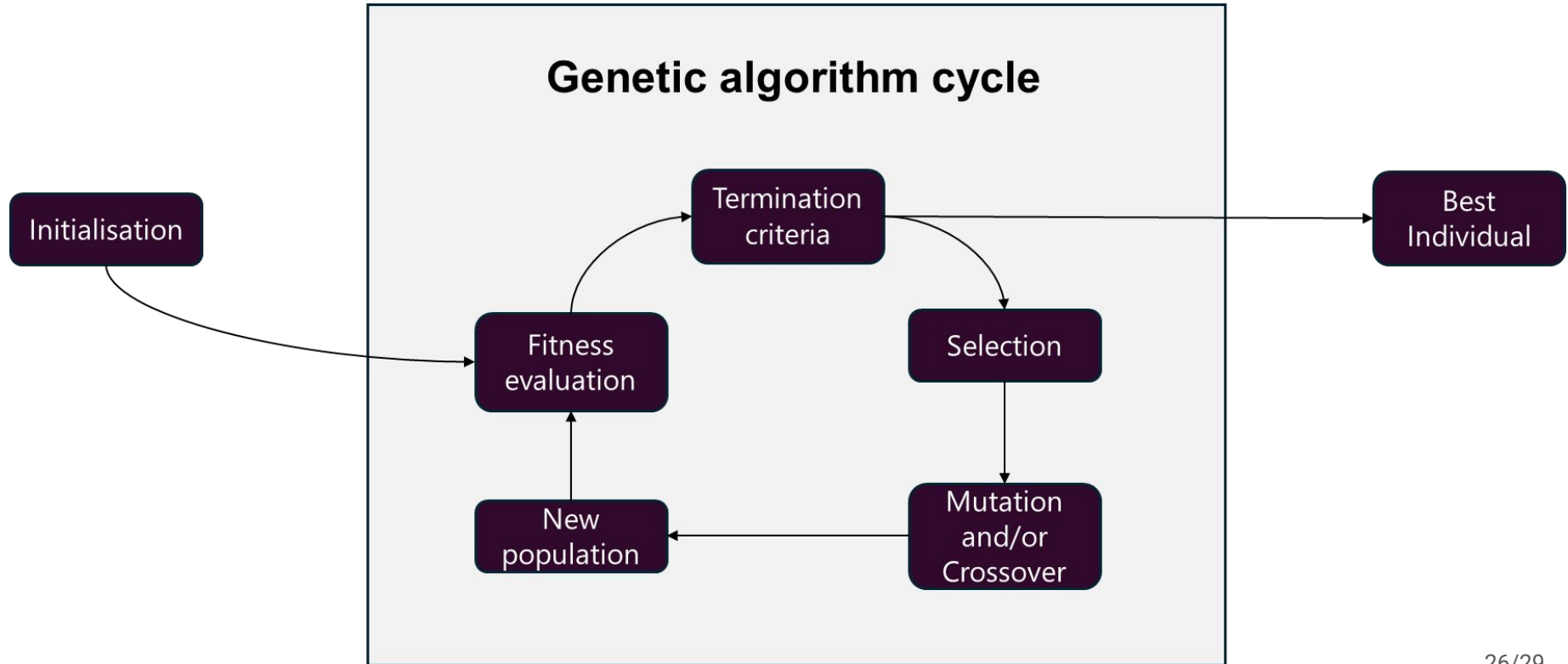
- **High Temp** (ex. 1.0 - 1.5)

The cat is on
the table

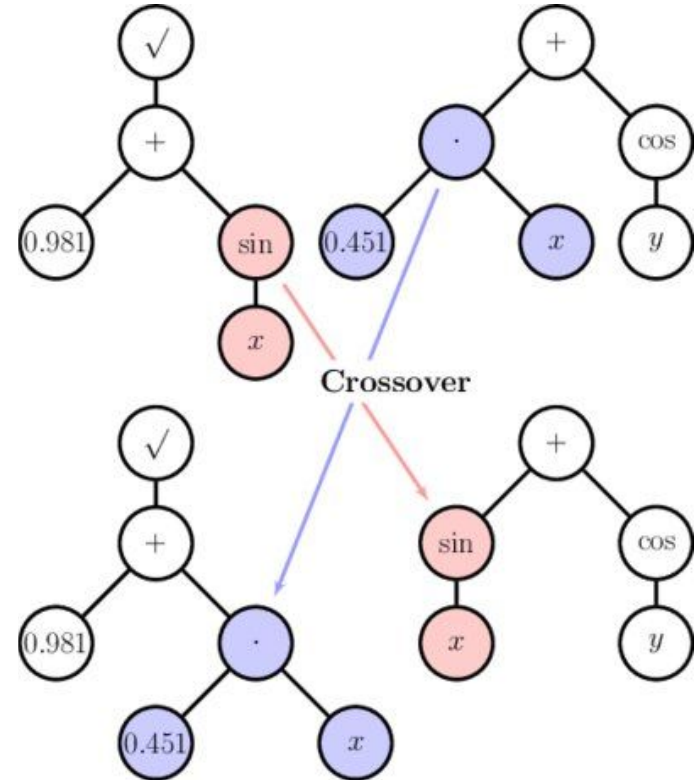
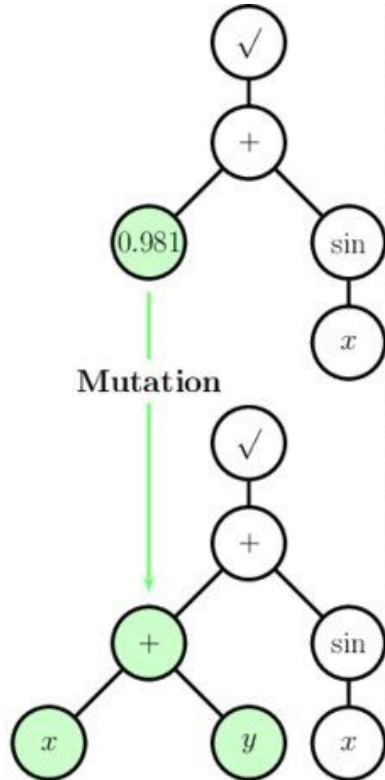


.
of
OMG
!

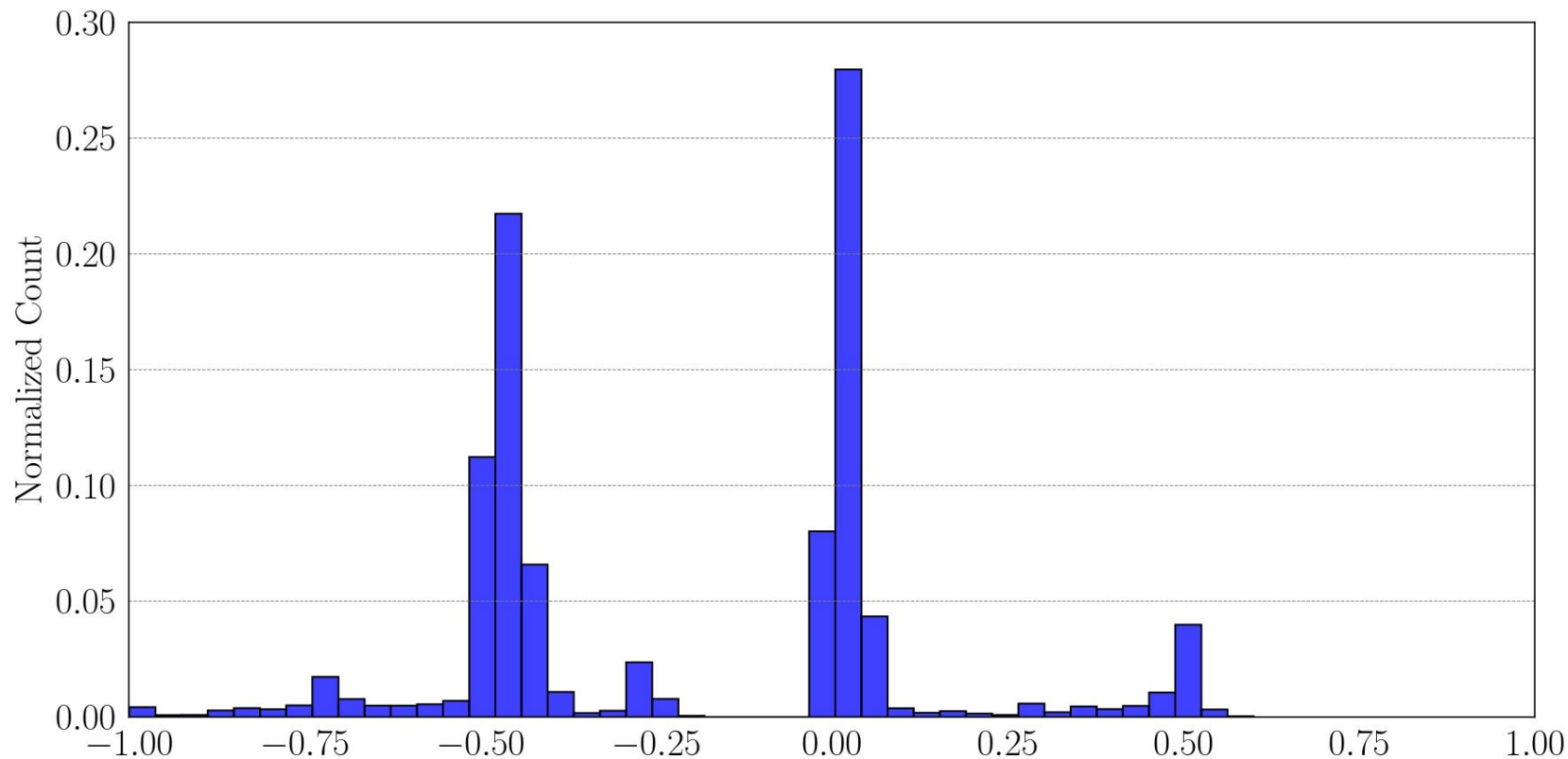
GA Cycle



Mutation & Crossover



QAS (EX - QAS)



QAS (Stability)

